СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ 4

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 5

ВВЕДЕНИЕ 6

ГЛАВА 1 8

1.1 Анализ существующих курсов по машинному обучению 8

1.1.1 Машинное обучение на ФКН НИУ ВШЭ 8

1.1.2 Машинное обучение на ВМК МГУ 9

1.1.3 Deep Learning School МФТИ 9

1.1.4 Open ML Course: Классические модели ML 10

1.1.5 Курс машинного обучения girafe-ai 10

1.1.6 Karpov.Courses: Инженер машинного обучения 11

1.1.7 Яндекс Практикум: ML-инженер с опытом 11

1.1.8 Обобщение результатов анализа 12

1.2 Анализ инструментов для автоматической проверки 14

1.2.1 nbgrader 15

1.2.2 Otter-Grader 15

1.2.3 CodeGrade 16

1.2.4 Яндекс.Контест 16

1.2.5 Stepik 17

1.2.6 GitHub Classroom 17

1.2.7 Kaggle InClass 18

1.2.8 PyEvalAI 18

1.2.9 Обобщение результатов анализа 19

1.3 Анализ подходов к оценке качества с помощью БЯМ 21

1.3.1 Подходы к использованию БЯМ в роли оценщика 21

1.3.2 Систематические смещения и ограничения 22

1.3.3 Критерии выбора БЯМ и инфраструктура доступа 23

1.3.4 Метрики оценки качества работы БЯМ-оценщика 25

1.3.5 Обобщение 26

1.4 Анализ подходов оценивания репозиториев с помощью агентов 26
1.4.1 Ограничения существующих средств автоматизации ... 27
1.4.2 Агентный подход к оцениванию репозиториев 27
1.4.3 Выбор БЯМ для агентной системы 27
1.4.4 Метрики оценки качества агентной системы 28
1.5 Формулировка гипотезы и образ решения 29
1.5.1 Формулировка гипотезы 29
1.5.2 Образ решения 31

ГЛАВА 2 33
2.1 Рабочая программа дисциплины 33
2.1.1 Пререквизиты 33
2.1.2 Разделы 33
2.1.3 Темы 33
2.1.4 Оцениваемые задания 34
2.1.5 Результаты обучения 36
2.2 Дизайн эксперимента по оценке БЯМ-оценщика 37
2.2.1 Объект оценивания и подход 37
2.2.2 Метрика качества 38
2.2.3 Формирование датасета 38
2.2.4 Эксперименты 41
2.3 Дизайн эксперимента по оценке агентной системы проверки проектов 42
2.3.1 Объект оценивания и подход 42
2.3.2 Метрика качества 44
2.3.3 Формирование датасета 44
2.3.4 Эксперименты 45
2.4 Итоги 46

ГЛАВА 3 47
3.1 Архитектура автоматизированной проверки в nbgrader 47
3.1.1 Требования к работе 47
3.1.2 Реализация расширения nbgrader 48
3.1.3 Создание задания преподавателем 48
3.1.4 Генерация студенческой версии 48

3.1.5 Конфигурация и запуск проверки 49
3.2 Архитектура агентной системы проверки проектных репозиториев 50

ЗАКЛЮЧЕНИЕ 51

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 51

А ПРОМПТЫ ДЛЯ ФОРМИРОВАНИЯ ДАТАСЕТА 57

Б КРИТЕРИИ ОЦЕНИВАНИЯ ПРОЕКТА 67

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

БЯМ — большая языковая модель

ВМК — факультет вычислительной математики и кибернетики

ДЗ — домашнее задание

ИИ — искусственный интеллект

КР — контрольная работа

МО — машинное обучение

МГУ — Московский государственный университет

МФТИ — Московский физико-технический институт

НИУ ВШЭ — Национальный исследовательский университет «Высшая школа экономики»

ФКН — факультет компьютерных наук

CV — Computer Vision (компьютерное зрение)

CLI — Command Line Interface (командная строка)

HTML — Hyper Text Markup Language (язык разметки гипертекста)

DLS — Deep Learning School

GPU — графический процессор

JN — Jupyter Notebook

LMS — Learning Management System (система управления обучением)

ML — Machine Learning (машинное обучение)

ODS — Open Data Science

NLP — Natural Language Processing (обработка естественного языка)

RecSys — recommender systems (рекомендательные системы)

SQL — Structured Query Language (язык структурированных запросов)

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Автоматическая проверка — оценивание учебных заданий программной системой без непосредственной ручной проверки преподавателем

Большая языковая модель (БЯМ) — нейросетевая модель, генерирующая текст

Инференс — процесс использования обученной модели искусственного интеллекта для обработки новых данных и получения предсказаний или результатов

Квантизация — процесс преобразования модели в более компактный формат с сохранением качества, уменьшая размер модели и требования к памяти

Кросс-проверка — взаимная проверка работ обучающимися по заданным критериям

Ноутбук — файл с расширением .ipynb для работы в Jupyter Notebook

Открытый теоретический вопрос — задание, требующее развёрнутого текстового ответа с объяснением, интерпретацией или выводом

Промпт — инструкция для БЯМ, задающая её поведение и результат

Рубрика — категория критериев качества, используемая для оценивания работы

Юнит-тест — автоматическая проверка отдельного фрагмента программы по заранее заданным условиям

Ячейка — блок кода в Jupyter Notebook, который может быть выполнен и проверен

Chain-of-Thought (CoT) — цепочка рассуждений БЯМ для решения задачи

Jupyter Notebook — интерактивная вычислительная среда, объединяющая код, текстовые пояснения и результаты выполнения в одном документе

ВВЕДЕНИЕ

Машинное обучение за последнее десятилетие стало одной из ключевых компетенций для IT-специалистов. Спрос на образовательные программы по данному направлению неуклонно растет: курсы предлагают ведущие университеты (НИУ ВШЭ, МГУ, МФТИ), корпоративные школы (ШАД Яндекса, Т-Образование) и открытые платформы (ODS, Stepik, Karpov.Courses). Вместе с ростом числа слушателей обостряется проблема масштабируемости оценивания: преподаватели тратят значительную часть времени на проверку работ, а при увеличении потока студентов качество обратной связи снижается.

Анализ существующих курсов по машинному обучению показал, что автоматизация оценивания остается фрагментарной. Существующие инструменты - юнит-тесты в Jupyter Notebooks, контестные системы, тесты с выбором ответа - эффективны для проверки кода и формализованных ответов, но не позволяют оценивать задания открытого типа: теоретические вопросы, развернутые объяснения, интерпретацию результатов. Такие задания по-прежнему проверяются вручную, что создает узкое место в образовательном процессе. Появление больших языковых моделей (БЯМ) открывает возможность автоматизировать проверку заданий, требующих понимания естественного языка и оценки содержательной корректности ответов.

Новизна работы заключается в разработке и апробации формата образовательного курса по машинному обучению, в котором автоматическая оценка открытых ответов реализована с использованием БЯМ, интегрированного в существующий учебный пайплайн nbgrader. В отличие от решений, ориентированных только на проверку кода и закрытых тестов, предложенный подход объединяет новый формат курса, встроенный БЯМ и его применение на реальной аудитории студентов.

Практическая значимость работы определяется внедрением разработанного курса в образовательный процесс НИУ ВШЭ, а также апробацией системы автоматической проверки на реальной аудитории студентов, что позволяет оценить её применимость и эффективность в условиях учебного процесса и

потенциальную переносимость на другие технические дисциплины.

Целью данной работы является разработка автоматизированной системы проверки учебных заданий на основе больших языковых моделей для курса по машинному обучению, ее интеграция в образовательный процесс и апробация.

Для достижения данной цели были поставлены следующие задачи:

1) Провести анализ существующих курсов по машинному обучению и сформировать профиль целевой аудитории.
2) Исследовать подходы к автоматизированной проверке учебных заданий в образовательной среде.
3) Разработать методологию преподавания и структуру учебного курса по машинному обучению.
4) Разработать контрольные задания, критерии оценивания и учебно-методические материалы курса.
5) Реализовать автоматизированную систему проверки учебных заданий с использованием больших языковых моделей.
6) Провести апробацию разработанного курса и системы автоматической проверки в образовательном процессе НИУ ВШЭ и проанализировать полученные результаты.

Критериями успешности решения выступают: сокращение времени проверки учебных заданий, высокая степень согласованности автоматических оценок с экспертными, а также положительная оценка скорости и качества обратной связи со стороны студентов по итогам апробации.

Работа состоит из введения, трех глав и заключения. В первой главе описана предметная область, проведен анализ существующих курсов и подходов к автоматизированной проверке, сформулированы требования к решению. Во второй главе представлена архитектура разработанного курса и системы автоматической проверки, описаны методология, структура заданий и механизм оценивания на основе БЯМ. В третьей главе приведены результаты апробации курса, анализ метрик качества автоматической проверки и обратная связь от студентов.

ГЛАВА 1

1.1 Анализ существующих курсов по машинному обучению

Для формирования методологической и содержательной основы разрабатываемого курса был проведён анализ семи образовательных программ по машинному обучению. Поскольку область активно развивается, при отборе приоритет отдавался актуальным и востребованным программам с открытыми материалами, допускающими содержательный анализ системы контроля знаний.

Выборка охватывает три сегмента. Первый — академические курсы ведущих российских университетов (НИУ ВШЭ, МГУ, МФТИ): они задают стандарт теоретической глубины и отражают опыт, с которым сталкивается студент профильной специальности. Второй — открытые образовательные инициативы сообществ (ODS, girafe-ai): они представляют альтернативу университетскому образованию для широкой аудитории без формального отбора. Третий — коммерческие онлайн-платформы (Karpov.Courses, Яндекс.Практикум): они отражают запросы рынка труда и ориентированы на практическую подготовку специалистов.

Для каждого курса рассматривались формат обучения, содержание программы, система контроля знаний и степень автоматизации процесса оценивания. Все анализируемые курсы ведутся на русском языке.

1.1.1 Машинное обучение на ФКН НИУ ВШЭ

Курс Е. А. Соколова [1] является одним из наиболее известных академических курсов по машинному обучению. Обучение проходит в очном формате для бакалавров факультета компьютерных наук НИУ ВШЭ, при этом все материалы, включая конспекты лекций и видеозаписи 2021 года, опубликованы в открытом доступе. Программа охватывает классические разделы машинного обучения.

Система контроля включает теоретические и практические домашние задания, контрольную работу и экзамен. Теоретические домашние задания и контрольные работы включают открытые вопросы с объяснением, а практические подразумевают работу в Jupyter Notebook. Домашние задания

принимаются через платформу Anytask - она может только собирать ответ студентов, при этом не поддерживает автоматизацию проверки.
Практические задания в Jupyter Notebook частично автоматизированы юнит-тестами, однако все практические и теоретические работы проверяют ассистенты. Ручная проверка таких заданий фиксируется как ключевое ограничение: при росте потока студентов она масштабируется только увеличением количества ассистентов.

1.1.2 Машинное обучение на ВМК МГУ

Курс машинного обучения для студентов третьего курса факультета вычислительной математики и кибернетики МГУ [2] по содержанию близок к программе НИУ ВШЭ. Обучение организовано в очном формате и сопровождается практическими и теоретическими видами контроля. В практические входит работа в Jupyter Notebook и соревнования на платформе Kaggle, а в теоретические - коллоквиумы, тесты на лекциях, открытые вопросы в лабораторных работах и экзамен.
Проверка практических заданий автоматизирована при помощи юнит-тестов, при этом некоторые задания требуют кросс-проверки между студентами в количестве трех человек [3].
Использование автотестов демонстрирует реализуемость автоматической проверки кода прямо внутри Jupyter Notebook. Кросс-проверка открытых ответов тремя студентами, напротив, отмечается как неудовлетворительный подход: она не обеспечивает согласованности оценок и перекладывает нагрузку на самих студентов.

1.1.3 Deep Learning School МФТИ

Школа глубокого обучения (DLS) при МФТИ [4] является бесплатным онлайн-курсом, реализованным на платформе Stepik. Программа разделена на четыре части. В первой части обсуждаются основы машинного обучения и нейронных сетей, во второй части - NLP, третья посвящена CV, а четвертая аудио.
Оценивание включает домашние задания в Jupyter Notebook, тесты, соревнования на платформе Kaggle и итоговый проект. Тесты автоматизированы на платформе Stepik. Домашние задания проверяются в формате кросс-валидации между студентами. Проект проверяется

преподавателем, закрепленным за студентом.

Курс имеет платный тариф, в котором предлагается доступ в приватный чат, персональный ментор, приоритетная проверка домашних заданий и дополнительные вебинары.

Кросс-проверка домашних заданий является ограничением — по тем же причинам, что и в курсе МГУ, при этом платный тариф показывает, что такой подход проверки может быть долгим. Также курс полностью не автоматизирован тестами и кросс-проверками: проект вручную проверяется преподавателем, а для более расширенной обратной связи необходимо перейти на платный тариф – это показывает важность обратной связи и подкрепляет сложность масштабирования на большую аудиторию студентов.

1.1.4 Open ML Course: Классические модели ML

Курс сообщества Open Data Science [5] представляет собой полностью онлайн-формат обучения и распространяется бесплатно без отбора слушателей. Темы включают линейные модели, решающие деревья и дополнительные главы по машинному обучению. Длительность курса составляет два месяца.

Проверочные контроли состоят из тестов, дополнительно дается возможность пройти соревнование. Проверка заданий полностью автоматизирована на собственной платформе ODS.ai.

Полная автоматизация проверки достигается здесь за счёт отказа от заданий открытого типа — контроль сводится к тестам и соревнованиям, поэтому этот компромисс фиксируется как нежелательный.

1.1.5 Курс машинного обучения girafe-ai

Открытый курс girafe-ai [6] содержит полный цикл изучения машинного обучения: от линейных моделей и метода опорных векторов до нейронных сетей, алгоритма обратного распространения ошибки, эмбеддингов и архитектур seq2seq. Материалы курса и видеозаписи лекций опубликованы в открытом доступе.

Контроль знаний организован через серию заданий и лабораторных работ в Jupyter Notebook, а также итоговый экзамен. Некоторые задания сразу включают тесты для самопроверки, а некоторые содержат теоретические вопросы. Нигде явно не описано, как именно и кем проверяются работы.

Теоретические вопросы и итоговый экзамен воспроизводят модель академических курсов и дополнительно подтверждают их подход: открытые задания необходимы для диагностики понимания алгоритмов, а не только их практического применения.

1.1.6 Karpov.Courses: Инженер машинного обучения

Коммерческий курс [7] реализован полностью в онлайн-формате. Программа ориентирована на подготовку специалистов от базового уровня до внедрения моделей в производственную эксплуатацию: линейные модели, деревья решений, нейронные сети, обработка естественного языка, рекомендательные системы, A/B-тестирование и развертывание моделей.

Классические семинары в курсе отсутствуют, практическая подготовка обеспечивается за счет решения прикладных задач на собственной платформе и разработки итогового проекта в формате ML-сервиса с проверкой экспертов.

Курс демонстрирует, что проектная модель оценивания с развёрнутой обратной связью является ключевой для подготовки специалистов. Полное отсутствие теоретических открытых вопросов рассматривается как ограничение: такой подход не позволяет диагностировать глубину понимания алгоритмов, что становится ограничением для академического курса.

1.1.7 Яндекс Практикум: ML-инженер с опытом

Курс «ML-инженер с опытом» от Яндекс Практикума [8] реализован полностью в онлайн-формате и рассчитан на четыре месяца обучения. Программа ориентирована на специалистов с базовыми знаниями классического машинного обучения и охватывает полный производственный цикл.

Система оценивания строится на пяти учебных проектах и итоговом проекте, выполняемом в условиях, приближённых к реальным рабочим задачам. Проверка проектов осуществляется ревьюерами с предоставлением развёрнутой письменной обратной связи. Практика выполняется во встроенной инфраструктуре платформы с доступом к Yandex Cloud. Про теоретические задания отдельно ничего не говорится. Студентам доступен Практикум ИИ - нейросеть, отвечающая на вопросы по материалу курса.

Курс подтверждает масштабируемость проектной модели в коммерческом сегменте, как и Karpov.Courses.

1.1.8 Обобщение результатов анализа

Результаты анализа систематизированы в трёх таблицах: в таблице 1.1 — общие характеристики и тематическое содержание программ, в таблице 1.2 — виды оценочных мероприятий, в таблице 1.3 — организация проверки учебных заданий.

Таблица 1.1 — Общая характеристика и тематическое содержание курсов

Курс	Тип	Формат	Год	Кл. МО	Гл. об.	Внедр.	Доп. темы
МО ФКН НИУ ВШЭ	Академический	Очный	2026	Да	—	—	—
МО ВМК МГУ	Академический	Очный	2026	Да	—	—	—
Open ML Course	Открытый	Онлайн	2026	Да	—	—	—
DLS МФТИ	Открытый	Онлайн	2026	Да	Да	—	NLP, CV, аудио
girafe-ai	Открытый	Очный	2021	Да	Да	—	seq2seq, эмбеддинги
Karpov.Courses	Коммерческий	Онлайн	2026	Да	Да	Да	A/B тесты
Яндекс Практикум	Коммерческий	Онлайн	2025	—	—	Да	RecSys, NLP
Кл. МО — классическое МО; Гл. об. — глубинное обучение; Внедр. — внедрение моделей в производство; «—» — отсутствует.							

Таблица 1.2 — Виды оценочных мероприятий

Курс	Практ. задания	Теор. задания	Тесты	Экз./КР	Проект	Соревн.
МО ФКН НИУ ВШЭ	Да (JN)	Да	Да	Да	—	—
МО ВМК МГУ	Да (JN)	Да	Да	Да	—	Да
Open ML Course	—	—	Да	—	—	Да
DLS МФТИ	Да (JN)	Да	Да	—	Да	Да
girafe-ai	Да (JN)	Да	Да	Да	—	—
Karpov.Courses	Да (плат.)	—	—	—	Да	—
Яндекс Практикум	Да (плат.)	—	—	—	Да	—
JN — Jupyter Notebook; плат. — собственная платформа курса; Соревн. — соревнования.						

Таблица 1.3 — Организация проверки домашних заданий

Курс	Платформа ДЗ	Автотесты	Кросс-пров.	Ручная пров.
МО ФКН НИУ ВШЭ	Anytask	—	—	Да
МО ВМК МГУ	Собственная	Да	Да	Да
Open ML Course	ODS.ai	Да	—	—
DLS МФТИ	Stepik, Kaggle	Да	Да	Да
girafe-ai	GitHub	Да	—	—
Karpov.Courses	Собственная	Да	—	Да
Яндекс Практикум	Собственная	—	—	Да

Проведённый анализ позволяет сформулировать ряд ключевых наблюдений. **Jupyter Notebook как стандарт практических заданий.** Четыре из семи рассмотренных курсов используют Jupyter Notebook в качестве основной

среды выполнения практических заданий. В ряде программ код в ноутбуках уже проверяется автоматически посредством юнит-тестов, а ответы на теоретические вопросы и выводы необходимо выписать в ячейках JN, что подтверждает принципиальную осуществимость автоматизации в данной среде и обозначает Jupyter Notebook как естественную точку интеграции для расширенной системы оценивания.

Классическое машинное обучение и теоретические задания. Классические алгоритмы машинного обучения образуют обязательное ядро всех семи программ вне зависимости от их типа и формата. В академических курсах изучение классических алгоритмов неизменно сопровождается теоретическими заданиями открытого типа — вопросами с развёрнутым ответом, интерпретацией результатов, формулировкой выводов, — которые направлены на диагностику глубины понимания математической природы алгоритмов, а не только умения применять их на практике. Задания открытого типа присутствуют в четырех из семи программ и характерны прежде всего для академических и открытых курсов.

Итоговый проект как маркер практической ориентации. Проект предусмотрен в трёх из семи курсов: в двух коммерческих (Karpov.Courses, Яндекс Практикум) и одном открытом (DLS МФТИ). Проект представляет собой многофайловый репозиторий с полным циклом работы: от разведочного анализа данных до обучения моделей, оформления отчёта и деплоя. Оценивание носит многокритериальный характер: качество предобработки, обоснованность выбора моделей и метрик, корректность экспериментов, оформление кода и документации. Примечательно, что именно коммерческие программы последовательно выстраивают обучение вокруг проекта как сквозной формы работы — это отражает запрос рынка труда на специалистов, способных решать комплексные задачи в производственных условиях. Вместе с тем оценивание проекта требует развёрнутой индивидуальной обратной связи от преподавателей или ревьюеров, что при увеличении числа студентов существенно нагружает преподавательский состав.

Проверка открытых заданий как ключевое узкое место. Оценивание теоретических заданий открытого типа и проектов сводится к трём подходам:

ручная проверка преподавателем, кросс-проверка между студентами или исключение подобных заданий из системы контроля. Каждый из них имеет существенные ограничения: ручная проверка не масштабируется при росте потока студентов, кросс-проверка не обеспечивает согласованности оценивания и перекладывает нагрузку на самих обучающихся, а отказ от открытых заданий снижает глубину диагностики знаний. Таким образом, автоматизация охватывает проверку кода и формализованных ответов, но практически не применяется к оценке текстовых ответов, требующих содержательного анализа. Это противоречие определяет необходимость рассмотрения существующих инструментов автоматической проверки.

1.2 Анализ инструментов для автоматической проверки

Задача автоматизации проверки учебных заданий решается различными инструментами: от классических систем юнит-тестирования Jupyter Notebook до коммерческих платформ и исследовательских прототипов на основе больших языковых моделей.

Для систематического сравнения введём критерии, существенные с точки зрения настоящей работы.

- **Поддержка Jupyter Notebook.** Анализ в 1.1 показал, что Jupyter Notebook является стандартом для практических заданий в курсах по машинному обучению, поэтому его поддержка является важным критерием.
- **Наличие открытых заданий.** Инструмент должен поддерживать создание открытых заданий, требующих развёрнутого текстового ответа с объяснением, интерпретацией или выводом.
- **Наличие автотестов.** Наличие механизмов юнит-тестирования кода или сверки с фиксированным правильным ответом.
- **Открытость кода.** Доступность исходного кода для изучения, модификации и расширения.
- **Интеграция с LMS.** Возможность встроить инструмент в существующий рабочий процесс преподавателя без значительных изменений инфраструктуры: поддержка CLI и/или отдельные LMS-интеграции.
- **Требования к инфраструктуре.** Необходимые аппаратные и программные ресурсы для развёртывания: наличие GPU, серверных мощностей и т. п.

– **Веб-интерфейс.** Наличие веб-интерфейса для просмотра результатов проверки и обратной связи с преподавателем оценивает порог входа для преподавателя.

Ниже рассмотрены восемь наиболее распространённых решений. Для каждого приведены краткая характеристика и явный вывод о применимости к автоматизации открытых заданий.

1.2.1 nbgrader

nbgrader — открытая система для создания и проверки заданий в формате Jupyter Notebook, разработанная в UC Berkeley и поддерживаемая сообществом Jupyter [9; 10]. Поддерживает два типа ячеек: автоматически проверяемые ячейки кода на основе assert-тестов и ячейки с открытым ответом. Открытые ответы — теоретические вопросы, текстовые объяснения — проверяются вручную через веб-интерфейс Jupyter Hub и расширения Formgrader. Проверка ячеек кода поддерживает скрытые тесты, которые не видны студенту.

Преподавателю необходимо разметить ячейки в Jupyter специальными комментариями, написать тесты и настроить структуру каталогов. Инструмент доступен как в JupyterHub, так и как консольная утилита – последнее позволяет интегрировать инструмент в существующий рабочий процесс преподавателя.

Отдельным плюсом инструмента является генерация отзыва студенту по результатам проверки в формате HTML. Для автоматически проверяемых ячеек он отображает только балл, а для открытых ответов помимо балла также и отзыв преподавателя.

Код открытый и библиотека активно поддерживается сообществом, что позволяет внедрять новую функциональность и исправлять найденные ошибки. Хотя кодовые ячейки все еще можно оценивать автоматически, проверка открытых ответов остается на ручном уровне, что создает дополнительные трудозатраты на преподавателя.

1.2.2 Otter-Grader

Otter-Grader — открытая система автоматической проверки на Python и R [11; 12]. Как и nbgrader, использует разметку ноутбука специальными комментариями. Ключевое отличие — модель исполнения: команда

`otter assign` генерирует из исходного ноутбука студенческую версию и архив для запуска проверки в изолированном Docker-контейнере, что упрощает масштабирование. Открытые вопросы помечаются как `manual: true` и экспортируются в PDF для ручной проверки во внешней системе. Собственного интерфейса для оценки открытых ответов нет.

Код открытый, однако сообщество и документация заметно меньше, чем у nbgrader. Зависимость от внешней системы при проверке открытых ответов означает, что автоматизация этого сценария в рамках самого инструмента не предусмотрена и потребовала бы значительной доработки.

1.2.3 CodeGrade

CodeGrade — коммерческая платформа, разработанная в Университете Амстердама [13]. Имеет нативную поддержку Jupyter Notebook, позволяющую визуально настраивать блоки тестов: проверки ввода-вывода, юнит-тесты, линтеры, структурные проверки и пользовательские скрипты. Интегрируется с LMS и включает встроенную проверку на плагиат. ИИ-ассистент для студентов, помогающий в решении задач, находится в стадии бета-тестирования. Код закрытый; продукт распространяется по подписке.

Открытые текстовые ответы автоматически не оцениваются — проверка остаётся ручной. Коммерческая модель и закрытый код исключают возможность расширения функциональности, что делает инструмент неприменимым для целей настоящей работы.

1.2.4 Яндекс.Контест

Яндекс.Контест — проприетарная платформа, реализующая классическую модель онлайн-судьи (stdin/stdout) [14]. Поддерживает программные задачи с тестами и пользовательские проверки. Jupyter Notebook не поддерживается; открытые теоретические вопросы оцениваться не могут.

Преподавателю необходимо подготовить условие задачи и набор тестов; настройка выполняется через веб-интерфейс платформы. Инструмент хорошо знаком студентам технических специальностей и активно применяется в олимпиадной практике.

Исходный код закрытый, платформа ориентирована исключительно на алгоритмические задачи. По этим причинам инструмент не применим в

контексте настоящей работы.

1.2.5 Stepik

Stepik — образовательная платформа, поддерживающая 18 типов автоматически проверяемых заданий: тесты с выбором ответа, численные задачи, проверку формул через SymPy, регулярные выражения, исполнение кода против тестов, SQL [15]. Для открытых ответов доступна кросс-валидация между студентами или проверка командой курса. Jupyter Notebook не поддерживается.

Преподаватель настраивает задания через веб-интерфейс; для нестандартных проверок можно написать функцию `check(reply)` на Python. Платформа предоставляет встроенную аналитику по результатам студентов и поддерживает сертификацию.

Исходный код закрытый. Jupyter Notebook не поддерживается — код выполняется во встроенном редакторе. Открытые ответы автоматически не оцениваются: кросс-валидация не обеспечивает согласованности оценок и перекладывает нагрузку на студентов, а ручная проверка командой курса не масштабируется. Отсутствие поддержки Jupyter Notebook в совокупности с невозможностью автоматической оценки открытых ответов делают Stepik неприменимым для целей настоящей работы.

1.2.6 GitHub Classroom

GitHub Classroom — бесплатный сервис для управления учебными заданиями на основе Git и GitHub Actions [16]. Каждое задание представлено шаблонным репозиторием; при каждом `git push` в Linux-окружении GitHub Actions запускаются преподавательские тесты, результаты которых отображаются в интерфейсе репозитория. Jupyter Notebook нативно не поддерживается, однако ноутбук может быть преобразован в Python-скрипт.

Преподавателю необходимо подготовить шаблонный репозиторий с тестами и настроить GitHub Actions; от студентов требуется базовое владение Git. Платформа удобно интегрируется с рабочим процессом разработки и приучает студентов к практике контроля версий, что особенно полезно для проектной работы.

Код инфраструктуры открытый. Jupyter Notebook нативно не поддерживается — ноутбук необходимо преобразовывать в Python-скрипт

для проверки. Открытые теоретические вопросы платформа не оценивает: проверка строится исключительно на автоматических тестах. Отсутствие нативной поддержки Jupyter и механизма оценки открытых ответов исключают использование инструмента в качестве основного решения.

1.2.7 Kaggle InClass

Kaggle InClass — режим закрытых учебных соревнований, позволяющий преподавателю создать соревнование с приватным тестовым набором [17]. Студенты загружают файлы предсказаний, которые автоматически оцениваются по выбранной метрике с публикацией результатов на лидерборде. Ни код, ни теоретические ответы платформа не анализирует.

Преподавателю необходимо подготовить датасет, разметку, скрытую часть ground truth и описание метрики.

Исходный код закрытый. Ни код решения, ни теоретические ответы платформа не анализирует — оценивается только качество предсказаний. Jupyter Notebook не поддерживается. Инструмент может служить полезным дополнением к основной системе оценивания, однако не решает задачу проверки открытых ответов и не может использоваться как самостоятельное решение.

1.2.8 PyEvalAI

PyEvalAI — закрытый исследовательский прототип, находящийся в стадии апробации [18]. Система оценивает как код, так и текстовые ячейки в Jupyter Notebook, комбинируя юнит-тесты с локально развёрнутой открытой БЯМ. Модель получает на вход условие задачи, эталонное решение, ответ студента и результаты тестов, после чего генерирует численную оценку и персонализированную обратную связь.

Преподавателю необходимо подготовить ноутбуки с заданиями и эталонными ответами, а также развернуть сервер с GPU для работы языковой модели. Инструмент доступен только через веб-интерфейс, консольная утилита отсутствует.

PyEvalAI в отличие от других инструментов поддерживает и автоматическую проверку кодовых ячеек с помощью тестов, и проверку открытых ответов с помощью БЯМ. Однако закрытый код не позволяет изучить, расширить или адаптировать систему под конкретный учебный процесс, а требование

GPU-сервера создаёт высокий технический порог входа и делает инструмент неприменимым там, где подобная инфраструктура недоступна. Также отсутствие CLI интерфейса исключает возможность интеграции с другими инструментами преподавателя.

1.2.9 Обобщение результатов анализа

Как видно из таблицы 1.4, все рассмотренные инструменты поддерживают автоматическую проверку закрытых заданий: в разных формах реализованы юнит-тесты для кода, сверка с эталонным ответом и соревновательная оценка предсказаний. Однако при переходе к открытым теоретическим ответам возможности большинства решений резко сужаются: семь из восьми инструментов либо перекладывают такую проверку на преподавателя или студентов через кросс-валидацию, либо вовсе не поддерживают этот тип заданий. Аналогичный вывод следует и по критерию поддержки Jupyter Notebook: нативная интеграция присутствует только у половины рассмотренных систем, тогда как в остальных случаях требуется преобразование ноутбука в скрипт или отказ от этого формата.

Таблица 1.4 — Сравнение инструментов автоматической проверки по базовым критериям

Инструмент	**JN**	**Откр. ответы**	**Автотесты**	**Открытый код**
nbgrader	Нативная	Только вручную	Да	Да
Otter-Grader	Нативная	Только вручную	Да	Да
CodeGrade	Нативная	Только вручную	Да	Нет
PyEvalAI	Нативная	Да, через БЯМ	Да	Нет
Яндекс.Контест	Нет	Нет	Да	Нет
Stepik	Нет	Вручную + кросс-валидация	Да	Нет
GitHub Class-room	Нет	Нет	Да	Нет
Kaggle InClass	Нет	Нет	Да	Нет

Единственным инструментом, сочетающим нативную Jupyter-интеграцию с автоматической оценкой открытых ответов через БЯМ, является PyE-

valAI. Однако данные таблицы 1.5 показывают, что это преимущество сопровождается существенными ограничениями: система не интегрируется с LMS и требует более сложной инфраструктуры, включая GPU-сервер. Дополнительно непубликуемый исходный код делает её непригодной в качестве основы для адаптации под конкретный учебный процесс. Поэтому PyEvalAI подтверждает принципиальную реализуемость подхода, но не может служить готовым решением.

Наиболее подходящей основой для расширения оказывается nbgrader. По базовым критериям он сочетает открытый исходный код, нативную поддержку Jupyter Notebook и автоматическую проверку программных заданий, а по критериям интеграции допускает как работу через CLI, удобную для встраивания в другие системы, включая академические LMS, так и использование веб-интерфейса Formgrader без обязательной LMS-интеграции. Таким образом, единственный незакрытый для nbgrader сценарий - автоматическая оценка открытых теоретических вопросов - не требует замены всей инфраструктуры: достаточно добавить новый тип ячейки с проверкой через БЯМ, сохранив привычный рабочий процесс преподавателя.

Таблица 1.5 — Сравнение инструментов автоматической проверки по критериям интеграции

Инструмент	**Интеграция с LMS**	**Веб-интерфейс**	**Требования к инфраструктуре**
nbgrader	Возможна через CLI	Да	Низкие
Otter-Grader	Возможна через CLI	Нет	Низкие
CodeGrade	Возможна	Да	Отсутствуют
PyEvalAI	Отсутствует	Да	Средние (GPU)
Яндекс.Контест	Отсутствует	Да	Отсутствуют
Stepik	Возможна	Да	Отсутствуют
GitHub Class-room	Возможна	Да	Отсутствуют
Kaggle InClass	Отсутствует	Да	Отсутствуют

1.3 Анализ подходов к оценке качества с помощью БЯМ

Использование больших языковых моделей в качестве автоматических оценщиков (подход, известный в англоязычной литературе как LLM-as-a-Judge) получило широкое распространение благодаря масштабируемости, воспроизводимости и способности анализировать ответы на естественном языке. Однако применение БЯМ для оценки учебных работ требует понимания возможностей и ограничений таких систем, выбора подходящей модели и инфраструктуры для её использования. В настоящем разделе рассматриваются три взаимосвязанных аспекта: методы оценки на основе БЯМ, систематические смещения таких оценщиков и критерии выбора модели и инфраструктуры.

1.3.1 Подходы к использованию БЯМ в роли оценщика

Парадигма LLM-as-a-Judge была систематизирована в основополагающей работе Zheng и соавторов [19], где авторы показали, что GPT-4 в роли оценщика достигает согласованности с экспертными оценками свыше 80 %, что соответствует уровню согласованности между самими экспертами.

Существует два основных класса методов: промпт-методы, использующие модели общего назначения, и специализированные модели, обученные для оценивания.

Промпт-методы. Модель общего назначения получает на вход задание, оцениваемый ответ, критерии оценивания и, при необходимости, эталонный ответ, после чего возвращает численную оценку и текстовую обратную связь. К промпт-методам относится подход G-Eval [20], в котором модель генерирует цепочку рассуждений (Chain-of-Thought) перед выставлением оценки, что повышает согласованность с экспертами. Обзоры Gu и соавторов [21] и Li и соавторов [22] систематизировали стратегии повышения надёжности: использование структурированных рубрик, запрос цепочки рассуждений, применение нескольких запуском с усреднением, а также ротацию порядка ответов при парном сравнении. Ключевое преимущество промпт-методов — отсутствие необходимости дообучения: достаточно грамотно сформулировать промпт с критериями. Промпт-подход используется и в прототипе PyEvalAI [18].

Обученные модели-оценщики. Альтернативный подход — дообучение открытых моделей специально для задачи оценивания. Среди

наиболее известных: JudgeLM [23], обученная на парных сравнениях с использованием GPT-4 в качестве учителя; Prometheus [24], обученная на 100 000 примеров с пользовательскими рубриками и способная работать как с абсолютным, так и с парным оцениванием. Prometheus 2 достигает 72–85 % согласованности с экспертами на ряде бенчмарков [25].

Однако эмпирические исследования показали существенные ограничения обученных оценщиков. Работа, сравнивающая JudgeLM, PandaLM, Auto-J и Prometheus с GPT-4 [26], продемонстрировала, что обученные модели фактически функционируют как узкоспециализированные классификаторы: они показывают высокую точность на тех схемах оценивания, на которых обучались, но при переходе к другой схеме (например, от парного сравнения к абсолютной оценке) их качество резко падает.

Таким образом, для образовательного контекста, где задания разнообразны по формату и содержанию, промпт-подход с использованием модели общего назначения представляется более гибким и практичным решением, чем обучение специализированного оценщика.

1.3.2 Систематические смещения и ограничения

Несмотря на высокую согласованность с экспертами, БЯМ-оценщики подвержены систематическим смещениям, которые необходимо учитывать при проектировании системы проверки. Классификация в работе Son и соавторов [27] выделяет 12 типов смещений; ниже описаны наиболее релевантные для образовательного контекста.

Смещение к многословности (verbosity bias) — тенденция оценивать более длинные ответы выше, даже если они не содержат дополнительной полезной информации [19; 27]. В образовательном контексте это может приводить к завышению оценок за «раздутые» ответы. Митигация достигается за счёт явного указания в промпте критериев содержательности и штрафа за не относящуюся к делу информацию.

Смещение к собственным ответам (self-enhancement bias) — склонность модели выше оценивать ответы, похожие по стилю на те, которые она сама генерирует. Panickssery и соавторы [28] продемонстрировали, что GPT-4 демонстрирует статистически значимое смещение в пользу собственных ответов, причём природа этого эффекта связана с перплексией: модель предпочитает тексты с более низкой перплексией. Для образовательной

задачи это означает, что ответы, сгенерированные популярными БЯМ, могут быть стилистически близки к обучающим данным модели-оценщика, и получать несколько завышенные оценки; наличие эталонного ответа и структурированных критериев частично снижает этот эффект.

1.3.3 Критерии выбора БЯМ и инфраструктура доступа

Выбор конкретной модели для автоматической проверки определяется совокупностью факторов: качество оценивания, стоимость, скорость, доступность в РФ, возможность локального развёртывания и лицензионные ограничения.

Качество оценивания моделей. По состоянию на апрель 2026 года ландшафт БЯМ характеризуется быстрым сближением качества проприетарных и открытых моделей. Согласно лидерборду Artificial Analysis [29], ранжирующему более 100 моделей по композитному индексу интеллекта, верхние позиции занимают три проприетарные модели от компаний OpenAI, Anthropic и Google. Открытые модели не отстают и формируют конкурентноспособность: модели от Alibaba, Z AI, DeepSeek близки к оценкам проприетраных моделей. Дополнительно, компания OpenAI имеет открытую модель gpt-oss-120b, которая также входит в список одних из самых качественных, несмотря на свой размер. При этом немаловажным остается скорость и стоимость – чем меньше у модели активных параметров, тем быстрее она будет генерировать ответ.

Доступность в РФ. Прямой доступ к API OpenAI, Anthropic и Google из России ограничен: оплата российскими картами невозможна, а аккаунты могут блокироваться. Российские модели — YandexGPT (Яндекс) и GigaChat (Сбер) — доступны без ограничений, принимают оплату в рублях и хранят данные на территории РФ.

Агрегаторы API и стоимость. Для доступа к зарубежным моделям из России или открытым моделям при отсутствии GPU существуют сервисы-агрегаторы, предоставляющие единый API, совместимый с OpenAI SDK, и принимающие оплату российскими картами. Среди них: Polza.ai, Bothub и ProxyAPI. Агрегаторы взимают наценку к оригинальной стоимости токенов, однако позволяют использовать флагманские модели GPT, Claude и Gemini без прямого доступа к API провайдера. Замена агрегатора сводится к смене `base_url` и API-ключа в конфигурации, что обеспечивает независимость

от конкретного посредника. Среди трех упомянутых агрегаторов Polza.ai является наиболее популярным и имеет наибольший выбор моделей, сохраняет наименьшую наценку и дает доступ к открытым моделям, которые развернуты только на территории РФ.

Стоимость использования моделей предоставлена в таблице 1.6.

Таблица 1.6 — Стоимость использования БЯМ

Модель / способ доступа	**Провайдер**	**Стоимость в руб. за 1 млн токенов (ввод/вывод)**	**Проприетарная**
GPT-5.4	Polza.ai	95 / 776,33	да
GPT-5.4 Nano	Polza.ai	18,25 / 114,05	да
GPT-5.4 Mini	Polza.ai	68,43 / 410,6	да
gpt-oss-120b	Polza.ai	8,92 / 66,92	нет
Claude Opus-4.7	Polza.ai	446,12 / 2 230,61	да
Claude Haiku-4.5	Polza.ai	47,87 / 237,83	да
Gemini 3.1 Pro Preview	Polza.ai	68,63 / 480,44	да
YandexGPT Pro 5.1	Яндекс	800	да
GigaChat 2 Pro	Сбер	500	да
GigaChat 2 Max	Сбер	650	да
Qwen3.5-397b-a17b	Polza.ai	34,8 / 208,78	нет
Qwen3.6 Plus	Polza.ai	29,65 / 177,93	нет
GLM-5.1	Polza.ai	124,91 / 374,74	нет
DeepSeek V4 Flash	Polza.ai	12,77 / 25,55	нет
DeepSeek V4 Pro	Polza.ai	406,03 / 501,84	да

Самая дешевая модель — DeepSeek V4 Flash, далее следует Qwen3.6 Plus и gpt-oss-120b. Это подтверждает, что открытые модели намного дешевле проприетарных. При этом самую низкую цену среди проприетарных моделей имеет Claude Haiku-4.5 и GPT-5.4 Nano. Стоит отметить, что точные размеры пропритерных моделей неизвестны, но чем меньше модель, тем меньше ресурсов она требует, а значит является более дешевой. Российские модели остаются одними из самых дорогих, уступая только Claude Opus-4.7.

Локальное развёртывание. Открытые модели могут быть развёрнуты локально, что полностью устраняет зависимость от внешних сервисов и вопросы конфиденциальности данных. Однако для инференса моделей, сопоставимых по качеству с проприетарными, требуются значительные вычислительные ресурсы GPU, что ограничивает практическую

применимость в условиях образовательного учреждения. Квантизированные версии моделей (4-bit, 8-bit) снижают требования к аппаратному обеспечению, но могут приводить к деградации качества оценивания. В индустрии для инференса локальных моделей используется инструменты vLLM [30] и SGLang [31], также поддерживающие OpenAI SDK.

Итого, открытые модели составляют хорошую конкуренцию проприетарным на общих бенчмарках, а также по скорости и стоимости. Дополнительно, открытые модели могут быть развернуты локально для конфиденциальности данных, однако это потребует ресурса GPU. В разрабатываемой системе важна возможность быстро сменить провайдера или модель – это позволяет сделать OpenAI SDK, который стал стандартом API для проприетарных и открытых систем.

1.3.4 Метрики оценки качества работы БЯМ-оценщика

Для оценки того, насколько хорошо автоматический оценщик воспроизводит экспертные оценки могут использоваться следующие метрики.

Quadratic Weighted Kappa (QWK) — коэффициент, измеряющий степень согласованности между двумя оценщиками с учётом случайного совпадения и квадратичного штрафа за расхождение. Значения интерпретируются следующим образом: 0.0–0.20 — слабая согласованность, 0.21–0.40 — удовлетворительная, 0.41–0.60 — умеренная, 0.61–0.80 — существенная, 0.81–1.00 — почти полная. QWK является стандартной метрикой в исследованиях автоматического оценивания письменных работ.

Процент точного совпадения (Exact Match) — доля случаев, когда автоматическая и экспертная оценки полностью совпадают. Такую ошибку легко интерпретировать, однако при повышении шкалы, может теряться понимание, насколько модель близка к ответу.

Корреляция Спирмена и Пирсона — коэффициенты, измеряющие монотонную и линейную связь между автоматическими и экспертными оценками соответственно. Используются для оценки того, сохраняет ли автоматический оценщик ранжирование работ.

Средняя абсолютная ошибка (Mean Absolute Error, MAE) — среднее отклонение балла системы от балла эксперта в шкале категории. Такая ошибка легко интерпретируется, так как показывает отклонение в оцениваемых единицах.

1.3.5 Обобщение

Анализ подходов к использованию БЯМ в роли оценщика позволяет зафиксировать следующее:

– Промпт-подход с использованием модели общего назначения является наиболее гибким решением для образовательного контекста, где задания разнообразны по формату. Обученные модели-оценщики демонстрируют переобучение на схему оценивания и теряют качество при переносе.
– БЯМ-оценщики подвержены систематическим смещениям, которые частично митигируются структурированными критериями и эталонным ответом.
– Доступ из РФ к проприетарным и открытым моделям возможен через агрегатор Polza.ai — он содержит наибольшее количество моделей, имеет низкую наценку, а для открытых моделей предлагает возможность выбрать те, которые развернуты на территории РФ. Российские модели значительно дороже открытых и зарубежных, поэтому от них решено отказаться в дальнейшем.
– Конфигурируемость API-эндпоинта в разрабатываемом решении обеспечивает независимость от конкретной модели и инфраструктуры доступа, позволяя адаптировать решение к доступным ресурсам и бюджету.

1.4 Анализ подходов оценивания репозиториев с помощью агентов

Анализ курсов по машинному обучению из раздела 1.1.8 показывает, что итоговый проект чаще используется в коммерческих и открытых курсах, так как он требует ручной проверки со стороны преподавателя и подразумевает фидбек по его итогам. Ручная проверка проектов плохо масштабируется и ведёт к задержкам обратной связи [32–34]. Поддержание единых стандартов по нескольким критериям человек выполняет неустойчиво при больших объёмах [34]. На практике это выливается в две стратегии: личная проверка преподавателем, ограничивающая размер курса, либо кросс-проверка студентами, ставящая вопрос о квалификации оценивающих.

Данный раздел пытается проанализировать проблему проверки проектов и изучить способы ее автоматизации.

1.4.1 Ограничения существующих средств автоматизации

Классические автогрейдеры на основе тестов проверяют функциональную корректность отдельных программ и применяются на курсах программирования. Для проектных работ по машинному обучению этого недостаточно: единственной правильной реализации не существует, а существенная часть оценки приходится на содержательные аспекты — обоснованность инженерных решений, качество оформления, аккуратность экспериментов, — которые невыразимы через тесты [33].

БЯМ-оценщик из раздела 1.3 также неприменим к проектам по двум причинам. Во-первых, размер репозитория превышает практически комфортный объём контекста, а качество БЯМ деградирует с ростом контекстного окна [35]. Во-вторых, единый промпт смешивает самостоятельные категории рубрики (предобработка, моделирование, стиль кода, деплой) в одну усреднённую оценку, что снижает интерпретируемость.

1.4.2 Агентный подход к оцениванию репозиториев

Указанные ограничения преодолеваются переходом к агентной архитектуре, в которой языковая модель снабжается инструментами для работы с файловой системой и явно декомпозирует задачу. Такой подход стал стандартным для аудита кода, код-ревью и репозиторного анализа [35–37]. Ключевые свойства — избирательное чтение релевантных файлов, многошаговое рассуждение и декомпозиция оценки по категориям — непосредственно отвечают на ограничения, перечисленные выше.

Для проектных заданий это означает использование системы специализированных БЯМ-агентов: каждый агент отвечает за свою категорию рубрики, самостоятельно определяет, какие файлы прочитать, и возвращает структурированную оценку с комментариями. Координация и агрегация выносятся на уровень оркестратора.

1.4.3 Выбор БЯМ для агентной системы

Переход к агентной архитектуре меняет требования к модели. Если оценщик из раздела 1.3 решает одношаговую задачу, и качество коррелирует с общими показателями интеллекта [29], то агент должен корректно вызывать инструменты, поддерживать состояние рассуждения через десятки шагов и работать с длинным контекстом. Эти свойства слабо

предсказываются общими индексами, поэтому ранжирование строится по специализированным бенчмаркам.

Стандартом измерения агентных способностей в работе с кодом стал SWE-bench Verified [38] — 500 реальных GitHub-issue, где модель должна сгенерировать патч, проходящий скрытые тесты. Решение требует именно тех свойств, что нужны для проверки проектов.

Верхние позиции SWE-bench Verified занимают модели Anthropic и OpenAI с разрывом в 2–3 процентных пункта; Gemini 3.1 Pro Preview от Google стабильно третий с заметным отставанием [39]. Среди открытых моделей лидируют MiniMax и Alibaba, конкурируя с проприетарными.

Критерий конфигурируемости API из раздела 1.3 сохраняется: оба компонента используют OpenAI-совместимый SDK, что позволяет независимо выбирать модель для каждой подзадачи.

1.4.4 Метрики оценки качества агентной системы

Метрики из раздела 1.3 измеряют согласованность численной оценки оценщика с экспертной — этого достаточно для одношагового сценария, где модель возвращает скаляр. Агентная проверка устроена иначе по двум осям: помимо численного балла по каждой категории рубрики система генерирует развёрнутый текстовый фидбек, а сам процесс проверки многошаговый и потребляет существенные ресурсы. Бенчмарки агентных систем ориентируются именно на это разделение — например, SWE-bench Verified [38] ранжирует системы по бинарному resolve rate (доле задач, где сгенерированный патч проходит все скрытые тесты [40]), но это работает только потому, что тесты дают объективный сигнал. В задаче проверки проектов такого автоматического верификатора нет, поэтому метрики строятся по трём осям: согласованность оценок, качество фидбека и операционные характеристики — такое разделение принято в обзорах оценки агентных систем [41].

Согласованность оценок. Структура численного выхода агентной системы совпадает со структурой выхода БЯМ-оценщика (раздел 1.3) — скалярный балл, — поэтому применимы те же метрики (MAE, корреляции с экспертом).

Качество фидбека. Текстовый фидбек — ключевой результат проверки, и его ценность определяется тем, насколько утверждения о коде соответствуют действительности. БЯМ при работе с кодом склонны

генерировать утверждения, расходящиеся с реальным содержимым: ссылки на несуществующие файлы и функции, неверные параметры, придуманные API [42; 43]. В агентном сценарии проблема усиливается: фидбек содержит десятки утверждений, и галлюцинации могут проявляться не только на уровне текста, но и в самом ходе рассуждений агента — его действиях, ссылках на наблюдения и истории выполнения [44]. Поэтому фидбек оценивается по двум осям: соответствию заданному формату и фактической корректности утверждений. Применяется бинарная схема — любое выдуманное утверждение обнуляет оценку фидбека, — отражающая требование надёжности: на практике важно отсутствие галлюцинаций, а не их доля.

Операционные характеристики. Агентные бенчмарки кроме качества результата измеряют и характеристики самого процесса исполнения — стоимость, время, число шагов, — поскольку для одной и той же задачи агенты на одинаковом скоре могут различаться по этим параметрам в десятки раз [41; 45]. Лидерборд SWE-rebench [46] публикует среднюю стоимость одной задачи, отдельные работы фиксируют долю успешных прогонов, частоту тайм-аутов и зацикливаний на повторных вызовах инструментов как самостоятельные сигналы о надёжности [45]. Для системы проверки проектов важны те же три величины. Доля успешных прогонов характеризует устойчивость многошагового исполнения. Стоимость и время определяют масштабируемость на курс из десятков студентов: расход токенов агентной сессией на порядок выше, чем у одношагового оценщика (раздел 1.4.3). Среднее число вызовов инструментов на агента отражает эффективность навигации по репозиторию: высокий показатель при сопоставимом качестве оценок указывает на избыточное чтение файлов и резерв оптимизации промпта.

1.5 Формулировка гипотезы и образ решения

1.5.1 Формулировка гипотезы

Анализ существующих курсов по машинному обучению (раздел 1.1) выявил два ключевых противоречия. Во-первых, открытые теоретические задания принципиально важны для диагностики глубины понимания алгоритмов, однако их проверка не поддаётся автоматизации средствами

ни одного из рассмотренных инструментов, кроме PyEvalAI с его закрытым кодом и GPU-зависимой инфраструктурой (раздел 1.2). Во-вторых, итоговые проектные работы — неотъемлемая часть всех рассмотренных коммерческих программ — проверяются исключительно вручную: классические автогрейдеры, основанные на наборах тестов, не способны оценить содержательные аспекты ML-проекта (обоснованность выбора моделей, качество предобработки, оформление кода), а проверка многофайлового репозитория единичным запросом к БЯМ упирается в ограничения контекстного окна и не позволяет получить интерпретируемую многокритериальную оценку (раздел 1.4).

Вместе с тем анализ подходов к оценке на основе БЯМ (раздел 1.3) показал принципиальную реализуемость промпт-подхода для автоматического оценивания текстовых ответов и его пригодность для образовательного контекста, где задания разнообразны по формату. Параллельно в литературе по анализу кода устойчиво демонстрируется, что задачи репозиторного уровня успешно решаются автономными БЯМ-агентами, оснащёнными инструментами навигации по файловой системе и декомпозицией задачи по специализированным ролям (раздел 1.4).

На основании проведённого анализа формулируется следующая гипотеза: автоматизация оценочной деятельности на курсе по классическому машинному обучению достижима для обоих ключевых типов заданий — открытых теоретических вопросов в среде Jupyter Notebook и итоговых проектных репозиториев — при использовании подходов, соответствующих природе каждого типа: интеграции БЯМ-оценщика в инфраструктуру nbgrader для теоретических ответов и многоагентной системы с инструментами чтения репозитория для проектных работ. Получаемая в обоих случаях оценка должна быть согласованной с ручной проверкой преподавателя без дополнительных временных затрат с его стороны.

Проверка гипотезы предполагает разработку соответствующего программного решения для каждого из двух компонентов, создание курса по классическому машинному обучению и проведение апробации на реальной учебной аудитории.

1.5.2 Образ решения

Предлагаемое решение состоит из двух взаимодополняющих компонентов, покрывающих оба типа оценочных активностей курса.

Компонент 1: БЯМ-оценщик теоретических ответов в nbgrader

Первый компонент представляет собой расширение стандартного функционала nbgrader, выбранного в разделе 1.2 как наиболее подходящая основа: он сочетает открытый исходный код, нативную поддержку Jupyter Notebook, автоматическую проверку программных заданий через assert-тесты и возможность интеграции через CLI.

В текущем виде nbgrader поддерживает два типа ячеек: автоматически проверяемые ячейки с кодом и ячейки с открытым ответом, проверяемые вручную. Предлагается ввести третий тип — ячейки с открытым ответом, оцениваемые автоматически с привлечением БЯМ. Для каждой такой ячейки преподаватель указывает условие задания, эталонный ответ и критерии оценивания; модель получает эти данные вместе с ответом студента и возвращает числовую оценку. Такой подход позволяет автоматизировать оценку коротких теоретических вопросов, формулировок выводов и интерпретаций результатов — тех типов заданий, которые необходимы для глубокого понимания материала (раздел 1.1), но традиционно требуют ручной проверки.

Компонент 2: агентная система проверки проектных репозиториев

Второй компонент решает задачу автоматизированной проверки итоговых проектов курса, сдаваемых в виде репозиториев на GitHub Classroom через последовательность чекпоинтов. В отличие от теоретических ответов, проект является многофайловым артефактом с многокритериальной рубрикой, что исключает применение единичного БЯМ-оценщика (раздел 1.4). Поэтому для проектов разрабатывается система специализированных БЯМ-агентов: каждый агент отвечает за свою категорию рубрики (предобработка данных, моделирование, стиль кода, деплой), самостоятельно навигирует по репозиторию через набор инструментов чтения файлов и возвращает структурированную оценку с

комментариями. Итогом проверки служат численные баллы по критериям и сгенерированный текстовый фидбек для студента.

Независимость от конкретной модели и инфраструктуры

Как показано в разделе 1.3, доступ к проприетарным и открытым моделям из РФ обеспечивается через агрегаторы с API, совместимым с OpenAI SDK; смена модели или провайдера сводится к изменению конфигурируемого параметра `base_url`. Это делает оба компонента решения независимыми от конкретной модели и инфраструктуры, позволяя адаптировать их к доступным ресурсам и бюджету.

Курс для апробации

Для апробации разрабатываемых инструментов создаётся курс по классическому машинному обучению, охватывающий базовые разделы дисциплины: линейные модели, решающие деревья, ансамблевые методы и обучение без учителя. Тематическое содержание курса определено по результатам анализа существующих программ (раздел 1.1): классические алгоритмы образуют обязательное ядро всех семи рассмотренных курсов, а Jupyter Notebook является стандартной средой выполнения практических заданий. Структура домашних заданий предусматривает ячейки с кодом и открытые теоретические вопросы, проверяемые с привлечением БЯМ; в качестве итоговой работы предусмотрен индивидуальный проект, сдаваемый по чекпоинтам и проверяемый агентной системой. Апробация проводится на студентах НИУ ВШЭ, что позволяет оценить работу обоих компонентов системы в условиях реального учебного процесса.

ГЛАВА 2

2.1 Рабочая программа дисциплины

Тематическое содержание разрабатываемого курса определяется выводами раздела 1.1: классические алгоритмы машинного обучения образуют обязательное ядро всех семи рассмотренных программ, а Jupyter Notebook является стандартной средой для практических заданий. Соответственно, в курсе этот контур воспроизводится: программа сосредоточена на классических разделах ML, а домашние задания и семинарские задачи выполняются в Jupyter Notebook — среде, в которую впоследствии встраивается разрабатываемый БЯМ-оценщик.

2.1.1 Пререквизиты

Для успешного освоения курса студент должен владеть:

– основами программирования на Python (синтаксис, функции, классы, работа с библиотеками);
– линейной алгеброй (матрицы, векторы, операции над ними);
– математическим анализом (производные, градиент);
– основами теории вероятностей и математической статистики.

2.1.2 Разделы

Курс состоит из восьми тематических разделов, охватывающих основные направления классического машинного обучения:

1) Введение в машинное обучение. Инструменты: NumPy, Pandas.
2) Линейная регрессия и методы оптимизации.
3) Классификация: логистическая регрессия, метод опорных векторов.
4) Деревья решений.
5) Ансамблевые методы: случайный лес, градиентный бустинг.
6) Кластеризация.
7) Специализированная тема по выбору: обработка естественного языка, компьютерное зрение или рекомендательные системы.
8) Консультация по проекту и защита проектов.

2.1.3 Темы

В рамках каждого раздела рассматриваются следующие темы:

– Разведочный анализ данных, предобработка, feature engineering, работа с дисбалансом классов.
– Математические основы машинного обучения: функции потерь, методы оптимизации (градиентный спуск и его модификации: SGD, Momentum, Adam).
– Линейные модели: линейная и логистическая регрессия, метод наименьших квадратов.
– Метрики качества классификации и регрессии: Precision, Recall, F1-мера, ROC-AUC, PR-AUC, MSE, RMSE, MAE.
– Метод опорных векторов и ядровой трюк.
– Деревья решений для регрессии и классификации: критерии разбиения, переобучение, регуляризация.
– Ансамблевые методы: бэггинг, случайный лес, градиентный бустинг, bias-variance trade-off.
– Методы кластеризации и уменьшение размерности.

2.1.4 Оцениваемые задания

2.1.4.1 Домашние задания

На курсе предусмотрено три домашних задания, каждое рассчитано на две недели выполнения. Дедлайны строгие; работы, поданные после срока, не принимаются. Итоговая оценка за домашние задания — среднее арифметическое трёх работ.

– Домашнее задание 1. Работа с NumPy: векторизованные вычисления, операции над массивами.
– Домашнее задание 2. Градиентный спуск. Реализация с нуля классов VanillaGradientDescent, StochasticGradientDescent, MomentumDescent, Adam, а также класса LinearRegression с поддержкой L1/L2-регуляризации. Все реализации должны быть векторизованы. С использованием разработанных классов необходимо обучить модель, предсказывающую выбросы CO2. Датасет был самостоятельно сформирован из открытых источников. За прохождение определенных порогов целевой метрики студенты получали дополнительные баллы.
– Домашнее задание 3. Классификация текста. Предобработка текста, построение признаковых представлений, обучение и настройка

классических моделей (логистическая регрессия, SVM), анализ метрик классификации и интерпретация результатов на задаче определения автора текста (человек или нейросеть). Домашнее задание частично выполняется в формате соревнования — наивысшую оценку (10 баллов) могут получить только топ-10 студентов, получивших максимальное значение целевой метрики на валидационной выборке.

2.1.4.2 Активность на семинарах

В ходе каждого семинара студентам предлагаются бонусные задания и ответы на вопросы. За одно занятие можно получить не более четырёх отметок активности. Итоговая оценка за активность — среднее за все занятия семестра. Шкала перевода:

- 4 отметки — 10 баллов;
- 3 отметки — 7,5 балла;
- 2 отметки — 5 баллов;
- 1 отметка — 2,5 балла;
- 0 отметок — 0 баллов.

Вопросы на семинарах формулировались так, чтобы подтолкнуть студентов к самостоятельному выводу, не давая ответ напрямую. Примеры вопросов: влияние коэффициента регуляризации λ на поведение модели при его уменьшении или росте; причины отказа от масштабирования OneHot-закодированных признаков; интерпретация метрики R^2.

Часть заданий носила практический характер: студентам предлагалось написать код для обучения модели или провести эксперименты с гиперпараметрами — например, исследовать влияние параметров случайного леса на качество модели и OOB-score.

2.1.4.3 Проект

Проект выполняется индивидуально (командные работы допускаются в исключительных случаях по согласованию с преподавателем). Работа ведётся в репозитории GitHub Classroom и сдаётся поэтапно через четыре чекпоинта с последующей защитой.

Требования к проекту:

– датасет — не менее 10 000 строк и не менее 9 признаков;
– наличие чётко сформулированной задачи машинного обучения;
– уникальность: датасет не должен пересекаться с проектами других студентов;
– при использовании Kaggle — только задачи типа monetary, задачи Getting Started не допускаются.

Структура сдачи и максимальные баллы:

– CP0 (выбор проекта, регистрация в GitHub Classroom) — допускной этап;
– CP1 (предобработка данных, базовое моделирование, качество кода) — 25 баллов, проходной порог 13 баллов;
– CP2 (расширенное моделирование, перебор гиперпараметров, ансамбли) — 30 баллов, проходной порог 15 баллов;
– CP3 (отчёт, деплой через FastAPI или аналог) — 30 баллов, проходной порог 15 баллов;
– защита (презентация 5 минут, ответы на вопросы) — 15 баллов.

Не набравший проходной балл на текущем чекпоинте не допускается к следующему; итоговая оценка за проект формируется из уже накопленных баллов. Критерии выполнения каждой категории приведены в приложении Б.

2.1.5 Результаты обучения

По итогам курса студент:

– знает математические основы классических алгоритмов машинного обучения: линейные модели, деревья решений, ансамблевые методы, методы кластеризации;
– умеет реализовывать алгоритмы оптимизации (градиентный спуск и его модификации) с нуля на Python с использованием NumPy;
– умеет проводить полный цикл работы с данными: разведочный анализ, предобработку, feature engineering, построение и отбор моделей;
– умеет выбирать и обосновывать метрики качества для задач регрессии и классификации;
– имеет практические навыки работы с текстовыми данными и признаковыми представлениями;
– умеет организовывать воспроизводимый ML-проект: структура кода, линтеры, фиксация зависимостей.

2.2 Дизайн эксперимента по оценке БЯМ-оценщика

Прежде чем встраивать БЯМ-оценщик в инфраструктуру проверки заданий, необходимо эмпирически подтвердить, что выбранный подход действительно обеспечивает качество оценки, сопоставимое с экспертным. Это требование непосредственно вытекает из гипотезы, сформулированной в разделе 1.5.1: автоматическая оценка должна быть согласованной с ручной проверкой преподавателя без дополнительных временных затрат с его стороны.

2.2.1 Объект оценивания и подход

Объектом оценки является БЯМ-оценщик, на вход которому подаётся вопрос, ответ студента и рубрика, а на выходе ожидается балл по дискретной шкале. Естественной единицей анализа здесь служит одна пара «вопрос–ответ»: на каждой такой паре оценщик выставляет ровно один балл, который сопоставляется с эталонным.

Для оценивания ответов студентов выбрана дискретная шкала из четырёх уровней: 0, 1, 2, 3 балла. Нулевой балл соответствует пустому или полностью нерелевантному ответу; баллы 1, 2 и 3 описываются индивидуальными критериями для каждого вопроса. Такая шкала является достаточно подробной, чтобы отражать качественные различия между ответами разной полноты, и одновременно достаточно компактной, чтобы критерии можно было сформулировать однозначно.

Анализ подходов к LLM-as-a-Judge показал, что качество БЯМ-оценщика существенно зависит от модели, формата промпта и наличия структурированных критериев оценивания. Поэтому ключевая исследовательская задача настоящей главы — подобрать сочетание модели и промпта, обеспечивающее приемлемое качество оценивания на типах заданий, характерных для курса по классическому машинному обучению. Для этого необходимо: зафиксировать метрику качества, определить охват предметной области и типов вопросов, собрать датасет вопросов с эталонными ответами и рубриками, сгенерировать на этой основе модельные ответы студентов с заранее известными баллами и провести серию экспериментов с различными конфигурациями оценщика.

2.2.2 Метрика качества

В качестве метрики качества согласованности БЯМ-оценщика с экспертной оценкой используется доля точного совпадения (exact match, accuracy) — доля случаев, когда автоматически выставленный балл совпадает с эталонным. Выбор этой метрики обусловлен соображениями интерпретируемости: на дискретной шкале из четырёх значений доля точного совпадения непосредственно отражает, как часто оценщик ставит ровно тот балл, что и эксперт, и не требует дополнительного пояснения, в отличие от Quadratic Weighted Kappa или коэффициентов корреляции (раздел 1.3).

Дополнительно оценивается среднее время ответа, стоимость в рублях и доля неуспешных генераций. Эти метрики необходимы, так как помимо качества оценки важны показатели доступности и экономической эффективности системы.

2.2.3 Формирование датасета

2.2.3.1 Покрытие предметной области и типы вопросов

Поскольку разрабатываемый оценщик предназначен для конкретного курса, бенчмарк должен покрывать темы, действительно входящие в его программу. В качестве предметного контура зафиксированы семь тем, соответствующих разделам курса: регрессия, классификация, деревья решений, бустинг, бэггинг и случайный лес, кластеризация, уменьшение размерности.

Помимо тематического разнообразия учитывается и формат вопросов. На основе анализа теоретических заданий из рассмотренных курсов (раздел 1.1) выделены пять типов вопросов открытого типа, перечисленные в таблице 2.1. Эти типы охватывают характерные форматы теоретических вопросов в курсах по машинному обучению — от воспроизведения формул и определений до диагностики учебных ситуаций.

Сочетание семи тем и пяти типов даёт 35 пар «тема–тип». Чтобы каждая такая пара была представлена не единичным примером, а более устойчивой выборкой, для каждой пары формируется по три вопроса — то есть целевой объём датасета составляет 105 вопросов.

Таблица 2.1 — Типы вопросов открытого типа

Тип	**Описание**
Формула или определение	Вопрос требует записать формулу, дать точное определение или назвать ключевое понятие
Объяснение механизма	Вопрос требует объяснить, почему или как работает алгоритм или его компонент
Интерпретация результата	Вопрос содержит конкретный результат (коэффициент, метрику, кривую обучения) и требует его интерпретации
Сравнение или выбор	Вопрос требует сравнить два подхода, алгоритма или гиперпараметра и обосновать выбор одного из них
Анализ ситуации	Вопрос описывает учебную ситуацию (высокая ошибка на валидации, нестабильность предсказаний и т. п.) и требует диагностики и рекомендации

2.2.3.2 Генерация вопросов и эталонных ответов

Ручная подготовка 105 разнообразных вопросов с эталонными ответами и покритериальными рубриками является трудозатратной и плохо масштабируемой процедурой. Поэтому первичная генерация делегирована БЯМ, а контроль качества выполнен преподавателем вручную.

Чтобы снизить риск систематических смещений, обусловленных особенностями конкретной модели — в том числе self-enhancement bias, описанного в разделе 1.3, — генерация выполнялась шестью различными моделями: Claude Sonnet 4.5, Claude Sonnet 4.6, Claude Haiku 4.5, Gemma 3, Qwen 3 и GLM-5 turbo. Использование разных семейств моделей обеспечивает стилистическое и содержательное разнообразие формулировок.

Каждой модели передавался единый системный промпт с требованиями к вопросу, эталонному ответу и рубрике, а также пользовательский промпт с указанием темы, типа вопроса и количества генерируемых единиц. Полные тексты системного и пользовательского промптов, использованных при генерации, приведены в приложении А, разделе А.1. Системный промпт фиксирует следующие требования:

– вопрос требует развёрнутого текстового ответа, не сводится к тесту или

коду, конкретен и однозначен;
- уровень сложности соответствует студенту третьего курса технического вуза; вопросы не касаются базовых определений, а нацелены на более комплексные и неочевидные аспекты;
- эталонный ответ занимает 3–6 предложений, содержит ключевые понятия и при необходимости формулы в LaTeX-нотации, не содержит избыточной информации;
- рубрика состоит ровно из трёх описаний — по одному на каждый из ненулевых баллов 1, 2, 3; формулировки критериев конкретны («студент указал, что ...», а не «правильность ответа»);
- ответ возвращается строго в формате JSON.

В результате каждая из шести моделей сгенерировала по 35 вопросов с ответами и рубриками — всего 210 вопросов.

2.2.3.3 Ручной отбор и валидация

Сгенерированный массив прошел ручную валидацию преподавателем. На этом этапе исключались дубликаты и близкие переформулировки между моделями, вопросы с фактическими ошибками, чрезмерно простые или, наоборот, выходящие за рамки программы курса формулировки, а также рубрики, в которых критерии для разных баллов были сформулированы неразличимо. По итогам отбора в финальный датасет вошло 105 вопросов, равномерно покрывающих 35 пар «тема–тип» по три вопроса на пару.

Результирующий датасет служит основой для последующих этапов: на нём генерируются ответы студентов разного качества и проводится оценка БЯМ-оценщика.

2.2.3.4 Генерация ответов студентов разного качества

Для оценки качества БЯМ-оценщика на дискретной шкале необходимо иметь набор ответов, для каждого из которых известен «правильный» балл с точки зрения рубрики. Сбор реальных ответов студентов на этом этапе невозможен (курс ещё не запущен), а ручное написание ответов по 105 вопросам на каждый из ненулевых баллов трудозатратно. Поэтому ответы генерируются БЯМ, после чего валидируются преподавателем.

Для генерации использовалась открытая модель gpt-oss-120b. Модели передавался специализированный промпт, инструктирующий её отвечать на вопрос ровно на заданный балл: с использованием формулировки соответствующего критерия рубрики и без добавления деталей, тянущих ответ на более высокий уровень.

Дополнительно требовалось писать в стиле, характерном для студента, — без избыточной формальности, а для низких баллов — максимально просто и кратко. Для каждого из 105 вопросов сгенерировано четыре ответа: на 1, 2 и 3 балла — моделью; на 0 баллов — пустой ответ, имитирующий ситуацию, когда студент не ответил вовсе. Полные тексты промптов, использованных для генерации ответов студентов заданного качества, приведены в приложении А, разделе А.2.

После генерации все ответы прошли ручную валидацию: было проверено, действительно ли ответ соответствует целевому критерию и заслуживает заявленного балла, и при необходимости корректировались формулировки. Итоговый объём датасета — 420 пар «вопрос–ответ» с известными эталонными баллами, равномерно распределёнными по шкале 0–3 балла.

2.2.4 Эксперименты

Собранный датасет из 420 пар «вопрос–ответ» с эталонными баллами используется для сравнения конфигураций БЯМ-оценщика. Цель экспериментов — выбрать сочетание модели и промпта, обеспечивающее наилучшее значение accuracy при приемлемых стоимости и скорости работы.

2.2.4.1 Варьируемые параметры

В рамках настоящей работы варьируются два основных параметра конфигурации оценщика.

Модель-оценщик. В соответствии с анализом ландшафта БЯМ (раздел 1.3) рассматриваются как проприетарные, так и открытые модели разных уровней качества и стоимости. Для всех моделей используется единый OpenAI-совместимый клиент через агрегатор, что позволяет менять модель изменением одного параметра конфигурации.

Шаблон промпта. Сравниваются варианты промпта, различающиеся наличием или отсутствием следующих элементов: эталонного ответа во

входных данных оценщика; явного запроса цепочки рассуждений (Chain-of-Thought) перед выставлением балла; Базовая часть промпта — формулировка задания, ответ студента, рубрика и максимальный балл — остаётся неизменной во всех конфигурациях. Примеры промптов приведены в приложении А, разделе А.3.

2.2.4.2 Протокол эксперимента

Каждая конфигурация запускается на полном датасете из 420 пар. Для каждой пары оценщик получает на вход вопрос, ответ студента, рубрику и — в зависимости от конфигурации — эталонный ответ; на выходе ожидается число от 0 до 3 в формате JSON.
Качество конфигурации характеризуется значением accuracy — долей пар, на которых полученный балл совпадает с эталонным; параллельно фиксируются среднее время отклика и суммарная стоимость прогона.

2.2.4.3 Результаты

Сводные результаты экспериментов представлены в таблице 2.2. Для каждой конфигурации указаны accuracy на полном датасете, среднее время отклика, суммарная стоимость и доля неуспешных генераций.

2.3 Дизайн эксперимента по оценке агентной системы проверки проектов

Помимо БЯМ-оценщика теоретических ответов, разрабатываемая инфраструктура включает многоагентную систему проверки проектных репозиториев (раздел 1.4). Поскольку входные данные, шкала и архитектура у этой системы принципиально отличаются от БЯМ-оценщика для коротких ответов, для неё формулируется собственный дизайн эксперимента, описанный в настоящем разделе.

2.3.1 Объект оценивания и подход

Объектом оценки является система, на вход которой подаётся репозиторий студента и описание чекпоинта, а на выходе формируются: численные баллы и текстовый фидбек по каждому критерию рубрики и агрегированные баллы по категориям. Оценка по каждой категории выставляется отдельным БЯМ-

Таблица 2.2 — Сравнение конфигураций БЯМ-оценщика на датасете из 420 пар

Модель	**Промпт**	**Accuracy**	**Ср. время на запрос, с**	**Стоимость на запрос, руб.**	**Неуспешных, %**
GPT-5.4 Nano	без эталона	0,85	1,91	4,37	0,00
	с эталоном	0,85	1,91	4,37	0,00
GPT-5.4 Mini	без эталона	0,89	2,70	16,48	0,00
	с эталоном	0,89	2,70	16,48	0,00
gpt-oss-120b	без эталона	0,94	8,42	4,16	0,02
	с эталоном	0,94	8,42	4,16	0,02
Qwen3.5-122B-A10B	без эталона	0,94	13,31	127,06	0,00
	с эталоном	0,94	13,31	127,06	0,00
Qwen3.6-Plus	без эталона	0,94	18,25	127,00	0,00
	с эталоном	0,94	18,25	127,00	0,00
Claude Haiku 4.5	без эталона	0,94	3,16	36,02	0,00
	с эталоном	0,94	3,16	36,02	0,00
DeepSeek V4 Flash	без эталона	0,94	5,46	5,63	0,00
	с эталоном	0,93	6,90	6,17	0,00

агентом, который самостоятельно навигирует по репозиторию через набор инструментов.

В отличие от БЯМ-оценщика, для которого эталонной единицей сравнения служит балл на одну пару «вопрос–ответ», у агентной системы естественной единицей анализа является репозиторий целиком. Поэтому ключевые метрики качества определяются в первую очередь на уровне репозитория и категории, а не отдельного критерия.

Ключевая исследовательская задача — проверить, что агентная система способна выставлять баллы и формировать текстовый фидбек, согласованные с экспертной разметкой, при приемлемой операционной надёжности и стоимости. Для этого необходимо: зафиксировать набор метрик качества, собрать выборку реальных репозиториев с эталонной разметкой, прогнать систему в штатной конфигурации и измерить её поведение по выбранным осям.

2.3.2 Метрика качества

На основе анализа подходов к оценке агентной системы (раздел 1.4) формулируются следующие метрики качества:

Основная метрика — согласованность баллов системы с эталонными баллами. На каждой категории отдельно вычисляются средняя абсолютная ошибка — интерпретируется как среднее отклонение балла системы от балла эксперта в шкале категории.

Помимо численной оценки система формирует текстовый фидбек для студента, и его качество критично для практического использования. Фидбек оценивается по двум независимым осям:

- соответствие заданному формату (бинарная метрика 0/1) — структура и форма фидбека, соответствие заданному промпту;
- фактическая корректность утверждений (бинарная метрика 0/1) — доля содержательных утверждений в фидбеке, которые действительно соответствуют коду репозитория, в противовес галлюцинациям (упоминание несуществующих файлов или конструкций, приписывание свойств, которых в коде нет). Если хотя бы одно утверждение выдумано, то ставится 0.

Для оценки практической применимости фиксируются:

- доля успешных прогонов — запусков, в которых все агенты завершились без сбоев, превышения таймаутов и зацикливания на повторных вызовах инструментов;
- среднее время полной проверки одного репозитория;
- средняя стоимость прогона по тарифам используемого агрегатора БЯМ;
- среднее число вызовов инструментов на одного агента — показатель эффективности навигации по репозиторию.

2.3.3 Формирование датасета

В качестве бенчмарка используется выборка из 20 репозиториев студентов, сданных по чекпоинту CP1 и CP2 курса (раздел 2.1.4.3). Чекпоинты CP1 и CP2 выбраны как наиболее показательные из доступных: они охватывает все основные категории рубрики — предобработку данных, базовое моделирование и качество кода — и формируют достаточный объём содержательной работы для оценивания.

Для каждого репозитория из выборки выставляются численный балл по

каждому критерию чекпоинта CP1 и CP2. Далее эти баллы агрегируются в суммарный балл по каждой категории (сумма баллов по критериям данной категории) и суммарный балл за чекпоинты CP1 и CP2.

Дополнительно пишется фидбек по улучшению работы студента, в случае, если он набрал не максимальный балл. В данных экспериментах решено сфокусироваться на точках роста в репозитории студента, которые позволят ему набрать максимальный балл.

2.3.4 Эксперименты

Собранная выборка из 20 репозиториев с эталонной разметкой используется для оценки качества агентной системы в штатной конфигурации. Цель экспериментов — измерить значения выбранных метрик и зафиксировать, насколько система пригодна к практическому использованию в учебном процессе. При этом в экспериментах будут варьироваться различные БЯМ.

Архитектура агентной системы Архитектура агентной системы представляет собой набор агентов, каждый из которых отвечает за свою категорию рубрики. Согласно чекпоинтам, всего оценивается 3 категории: предобработка данных, базовое моделирование и качество кода. Каждый агент получает на вход специализированный промпт, который содержит критерии выполнения категории.

На вход каждому агенту подается:

- системный промпт;
- путь к локальной копии репозитория студента;
- список критериев выполнения категории, их уникальные идентификаторы и максимальное количество баллов за каждый критерий;
- инструменты для навигации по репозиторию;

Системный промпт включает в себя описание категории и требования к формату фидбека и является системным для каждого агента определенной категории. Примеры промптов приведены в приложении А, разделе А.4.

Агенту необходимо самостоятельно навигировать по репозиторию, выбирая инструменты (tools), а в конце вернуть JSON-объект с результатом оценки. JSON-объект должен содержать список, где каждый элемент содержит:

- уникальный идентификатор критерия;
- результат оценки по этому критерию;
- фидбек по улучшению работы студента по этому критерию;

Архитектура одного агента изображена на рисунке 2.1.

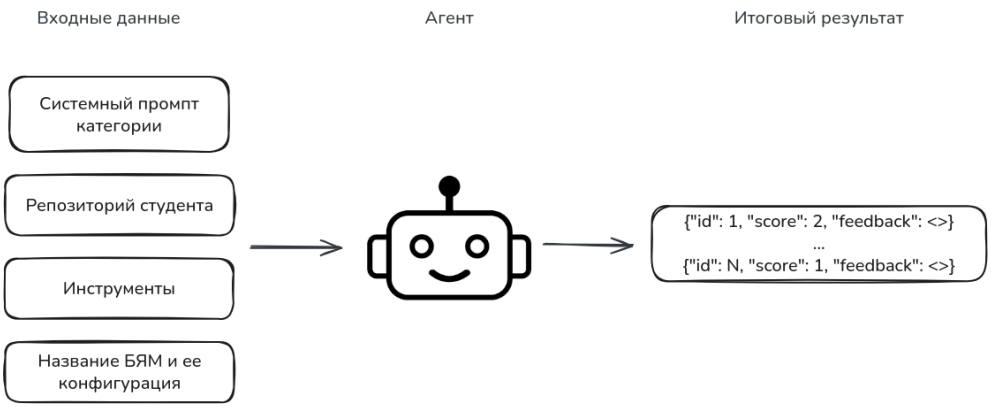

Рисунок 2.1 — Архитектура одного агента

Инструменты

Инструменты, доступные каждому агенту:

– чтение содержимого репозитория;
– чтение определенного файла в определенных строках;
– подсчет количества строк в файле;
– поиск паттерна в файле;
– чтение ноутбука (файл с расширением .ipynb);

Эти инструменты позволяют агенту самостоятельно навигировать по репозиторию и выполнять необходимые действия.

2.3.4.1 Результаты

2.4 Итоги

ГЛАВА 3

3.1 Архитектура автоматизированной проверки в nbgrader

В стандартной версии nbgrader для ячеек с кодом доступны следующие типы: Autograded answer - ячейка, в которой студент пишет код, проверяемый автоматическими assert-тестами; Autograded tests - ячейка с тестами, скрытая от студента и запускаемая при проверке; Manually graded answer - ячейка с кодом или текстом, требующая ручной оценки; Manually graded task - аналогичный тип, но оценка выставляется на уровне задания целиком; Read-only - ячейка, недоступная для редактирования студентом. Для текстовых (Markdown) ячеек набор типов ограничен: Manually graded answer, Manually graded task и Read-only. Таким образом, автоматическая проверка предусмотрена только для кода, а текстовые ответы могут быть оценены исключительно вручную.

3.1.1 Требования к работе

На основании проведенного анализа были сформулированы следующие требования к разрабатываемому решению.

Требования к новому типу ячейки:

1) В nbgrader должен быть добавлен новый тип ячейки - LLM graded answer, доступный для выбора наряду со стандартными типами.
2) Ячейка должна поддерживать структурированную разметку с разделителями: блок с формулировкой задания (видимый студенту) и блок с эталонным ответом и критериями оценивания (скрытый от студента).
3) Разделители должны быть настраиваемыми через конфигурацию.

Требования к генерации студенческой версии:

1) При генерации студенческой версии задания блок с критериями должен автоматически удаляться и заменяться текстом-заглушкой.
2) Формулировка задания должна сохраняться в неизменном виде.
3) Генерация должна выполняться стандартной командой nbgrader без дополнительных утилит.

Требования к конфигурации и промпту:

1) Преподаватель должен иметь возможность задать параметры подключения к API языковой модели: ключ, адрес, название модели.
2) Должен поддерживаться настраиваемый шаблон промпта с подстановкой переменных: формулировка задания, ответ студента, вывод ячейки, критерии оценивания, максимальный балл.
3) Должен быть задан таймаут ожидания ответа от API.

Требования к выходу модели и интеграции:

1) Модель должна возвращать численную оценку в диапазоне от 0 до максимального балла за задание.
2) Проверка ячеек типа LLM graded answer должна запускаться стандартной командой nbgrader autograde совместно с проверкой остальных типов ячеек.
3) Результаты проверки должны сохраняться в базу данных nbgrader, обеспечивая совместимость с существующим интерфейсом просмотра оценок.

3.1.2 Реализация расширения nbgrader

В рамках работы в инструмент nbgrader был добавлен новый тип ячейки - LLM graded answer, предназначенный для заданий, ответ на которые требует проверки языковой моделью. На рис. 3.1 показан пример выбора типа ячейки с добавленным вариантом.

3.1.3 Создание задания преподавателем

При подготовке преподавательской версии задания в Jupyter Notebook преподаватель выбирает тип ячейки LLM graded answer и заполняет её с помощью специальных разделителей, принятых в nbgrader:

– блок BEGIN QUESTION_LLM ... END QUESTION_LLM содержит формулировку задания, видимую студенту;
– блок BEGIN CRITERIA_LLM ... END CRITERIA_LLM содержит эталонный ответ и критерии оценивания, скрытые от студента.

Пример разметки ячейки приведён на рис. 3.2.

3.1.4 Генерация студенческой версии

Генерация студенческой версии задания выполняется стандартной командой nbgrader: nbgrader generate_assignment.

В результате блок с критериями удаляется, а на его месте появляется

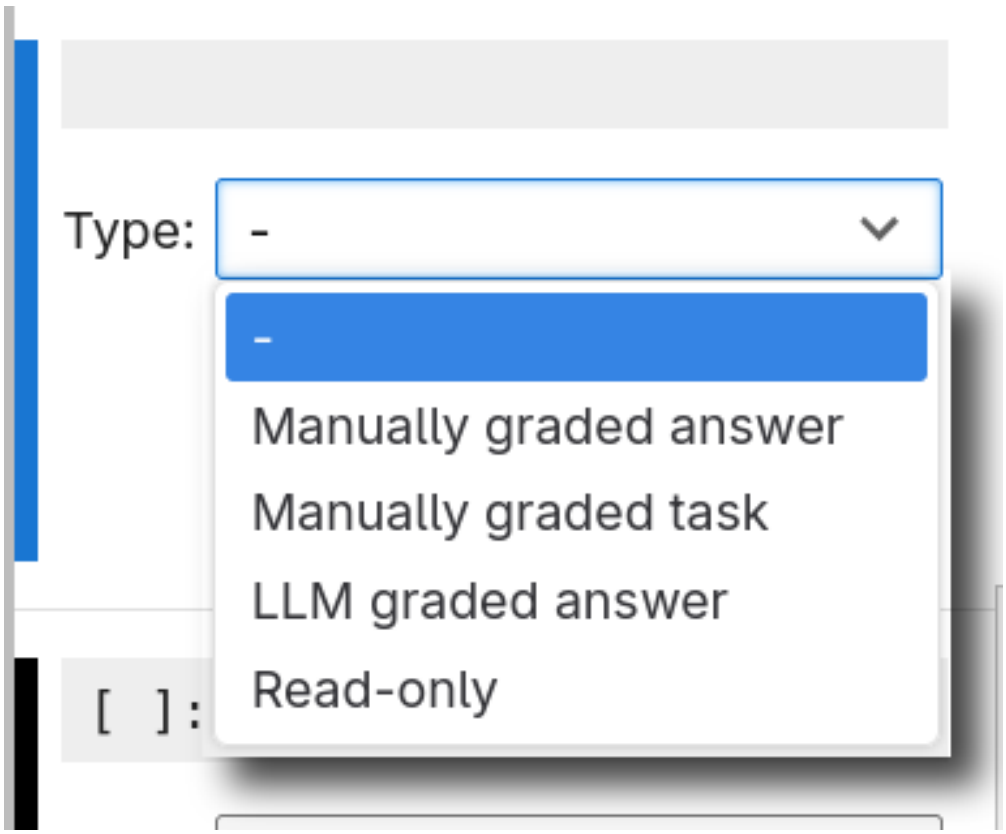

Рисунок 3.1 — Типы ячеек в nbgrader

текст YOUR ANSWER HERE, сохраняя при этом формулировку задания в неизменном виде. Пример отображен на рис. 3.3.

3.1.5 Конфигурация и запуск проверки

Перед запуском автоматической проверки преподаватель заполняет конфигурационный файл, задающий параметры обращения к БЯМ. Ключевые поля конфигурации:

- llm_api_key, llm_api_base, llm_model - параметры подключения к API языковой модели;
- БЯМ_prompt_template - шаблон промпта, в который подставляются:
  - {question} - формулировка задания;
  - {student_answer} - ответ студента;
  - {cell_output} - вывод ячейки (если есть);
  - {grading_instructions} - критерии оценивания;
  - {max_points} - максимальный балл за задание.
- begin_question_delimiter, end_question_delimiter и аналогичные поля для критериев - настройка разделителей разметки; по умолчанию равны BEGIN QUESTION_LLM и END QUESTION_LLM для заданий и BEGIN CRI-

Задание 2.1. Градиент MSE в матричном виде (0.04 балла).

BEGIN QUESTION_LLM

Напомним, что функция потерь MSE записывается как:

$$Q(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \langle x_i, w\rangle)^2 = \frac{1}{n}\|Xw - y\|^2$$

где n – количество объектов в выборке, $X \in \mathbb{R}^{n\times d}$ – матрица "объект-признак", а $y \in \mathbb{R}^n$ – целевая переменная. Через x_i обозначается i-ая строчка матрицы X, отвечающая за i-й объект выборки.

Выпишите ниже (подсмотрев в семинар или решив самостоятельно) градиент для функции потерь MSE в матричном виде.

END QUESTION_LLM

BEGIN CRITERIA_LLM

Эталонный ответ: Градиент MSE в матричном виде: $\|Xw - y\|^2$ по w: $2X^\top(Xw - y)/n$.

Критерии оценивания (4 балла):

- 4 балла: указана правильная формула $2X^\top(Xw - y)/n$ в различных записях (latex, markdown)
- 0 баллов: неверная формула или отсутствие ответа.

END CRITERIA_LLM

Рисунок 3.2 — Пример ячейки, заполненной преподавателем

Задание 2.1. Градиент MSE в матричном виде (0.04 балла). ¶

Напомним, что функция потерь MSE записывается как:

$$Q(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \langle x_i, w\rangle)^2 = \frac{1}{n}\|Xw - y\|^2$$

где n – количество объектов в выборке, $X \in \mathbb{R}^{n\times d}$ – матрица "объект-признак", а $y \in \mathbb{R}^n$ – целевая переменная. Через x_i обозначается i-ая строчка матрицы X, отвечающая за i-й объект выборки.

Выпишите ниже (подсмотрев в семинар или решив самостоятельно) градиент для функции потерь MSE в матричном виде.

YOUR ANSWER HERE

Рисунок 3.3 — Пример ячейки, которую видит студент

TERIA_LLM и END CRITERIA_LLM для критериев.

– llm_timeout - максимальное время ожидания ответа от API (в секундах).

Модель получает на вход перечисленные данные и возвращает число от 0 до max_points, представляющее выставленный балл.

После получения ответов студентов преподаватель запускает проверку стандартной командой: nbgrader autograde.

Система автоматически обходит все ячейки типа LLM graded answer, отправляет соответствующие запросы к языковой модели и сохраняет выставленные баллы в базу данных nbgrader. Остальные типы ячеек проверяются штатными механизмами инструмента.

3.2 Архитектура агентной системы проверки проектных репозиториев

ЗАКЛЮЧЕНИЕ

В заключении подведены итоги проведённого исследования. Разработанная методика применения алгоритмов глубокого обучения для классификации данных показала свою эффективность на тестовых наборах. Проведённый анализ литературы и экспериментальные исследования подтвердили, что предложенный подход позволяет снизить процент ошибок классификации по сравнению с традиционными методами.

Основные выводы работы:

– разработанная методика оправдана с точки зрения теоретических обоснований и практического применения;
– экспериментальные результаты демонстрируют преимущества глубоких нейронных сетей для задач классификации;
– работа содержит рекомендации для дальнейших исследований и внедрения метода в информационные системы.

Таким образом, проведённое исследование подтверждает перспективность использования методов глубокого обучения в решении актуальных задач обработки данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Соколов Е. А. Машинное обучение 1. Курс факультета компьютерных наук НИУ ВШЭ. — дата обращения: 14.04.2026. URL: https://github.com/esokolov/ml-course-hse.
2. Курс машинного обучения факультета ВМК МГУ. — дата обращения: 14.04.2026. URL: https://github.com/MSU-ML-COURSE.
3. Оценивание курса ВМК МГУ. — дата обращения: 15.04.2026. URL: https://github.com/MSU-ML-COURSE/ML-COURSE-25-26/blob/master/slides/intro/ML$_g rading_s ystem.pdf$.
4. Deep Learning School при ФПМИ МФТИ. — дата обращения: 14.04.2026. URL: https://dls.samcs.ru/.
5. Open ML Course: Классические модели ML. — дата обращения: 15.04.2026. URL: https://ods.ai/tracks/classic-models-spring26.
6. Открытый курс машинного обучения girafe-ai. — дата обращения: 14.04.2026. URL: https://github.com/girafe-ai/ml-course.
7. Karpov.Courses: Инженер машинного обучения. — дата обращения: 14.04.2026. URL: https://karpov.courses/ml-start.
8. Яндекс Практикум: ML-инженер с опытом. — дата обращения: 18.04.2026. URL: https://practicum.yandex.ru/machine-learning/.
9. nbgrader: A system for assigning and grading notebooks. — дата обращения: 14.04.2026. URL: https://nbgrader.readthedocs.io/en/stable/.
10. nbgrader: Github repository. — дата обращения: 14.04.2026. URL: https://github.com/jupyter/nbgrader.
11. Otter-Grader: A Python and R autograding solution. — дата обращения: 14.04.2026. URL: https://otter-grader.readthedocs.io/en/latest/.
12. Otter-Grader: Github repository. — дата обращения: 14.04.2026. URL: https://github.com/ucbds-infra/otter-grader.
13. CodeGrade: The Engaging Code Learning Platform. — дата обращения: 14.04.2026. URL: https://www.codegrade.com/.

14. Яндекс Контест — платформа для решения задач по программированию. — дата обращения: 14.04.2026. URL: https://contest.yandex.ru/.

15. Stepik: образовательная онлайн-платформа. — дата обращения: 14.04.2026. URL: https://stepik.org/.

16. Use autograding — GitHub Classroom Documentation. — дата обращения: 15.04.2026. URL: https://docs.github.com/en/education/manage-coursework-with-github-classroom/teach-with-github-classroom/use-autograding.

17. Kaggle Competitions. — дата обращения: 15.04.2026. URL: https://www.kaggle.com/competitions.

18. PyEvalAI: AI-assisted evaluation of Jupyter Notebooks for immediate personalized feedback / N. Wandel [et al.]. — 2025. — arXiv: 2502.18425 [cs.AI]. — URL: https://arxiv.org/abs/2502.18425.

19. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena / L. Zheng [et al.]. — 2023. — arXiv: 2306.05685 [cs.CL]. — URL: https://arxiv.org/abs/2306.05685.

20. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment / Y. Liu [et al.]. — 2023. — arXiv: 2303.16634 [cs.CL]. — URL: https://arxiv.org/abs/2303.16634.

21. A Survey on LLM-as-a-Judge / J. Gu [et al.]. — 2025. — arXiv: 2411.15594 [cs.CL]. — URL: https://arxiv.org/abs/2411.15594.

22. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods / H. Li [et al.]. — 2024. — arXiv: 2412.05579 [cs.CL]. — URL: https://arxiv.org/abs/2412.05579.

23. Zhu L., Wang X., Wang X. JudgeLM: Fine-tuned Large Language Models are Scalable Judges. — 2025. — arXiv: 2310.17631 [cs.CL]. — URL: https://arxiv.org/abs/2310.17631.

24. Prometheus: Inducing Fine-grained Evaluation Capability in Language Models / S. Kim [et al.]. — 2024. — arXiv: 2310.08491 [cs.CL]. — URL: https://arxiv.org/abs/2310.08491.

25. Prometheus 2: An Open Source Language Model Specialized in Evaluating Other Language Models / S. Kim [et al.]. — 2024. — arXiv: 2405.01535 [cs.CL]. — URL: https://arxiv.org/abs/2405.01535.

26. An Empirical Study of LLM-as-a-Judge for LLM Evaluation: Fine-tuned Judge Model is not a General Substitute for GPT-4 / H. Huang [et al.]. — 2025. — arXiv: 2403.02839 [cs.CL]. — URL: https://arxiv.org/abs/2403.02839.

27. Justice or Prejudice? Quantifying Biases in LLM-as-a-Judge / J. Ye [et al.]. — 2024. — arXiv: 2410.02736 [cs.CL]. — URL: https://arxiv.org/abs/2410.02736.

28. Wataoka K., Takahashi T., Ri R. Self-Preference Bias in LLM-as-a-Judge. — 2025. — arXiv: 2410.21819 [cs.CL]. — URL: https://arxiv.org/abs/2410.21819.

29. LLM Leaderboard — Comparison of over 100 AI models. — дата обращения: 19.04.2026. URL: https://artificialanalysis.ai/leaderboards/models.

30. Efficient Memory Management for Large Language Model Serving with PagedAttention / W. Kwon [et al.] // Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles. — 2023.

31. SGLang. — дата обращения: 01.05.2026. URL: https://github.com/sgl-project/sglang.

32. Akyash M., Azar K. Z., Kamali H. M. StepGrade: Grading Programming Assignments with Context-Aware LLMs // arXiv preprint arXiv:2503.20851. — 2025.

33. Autograder+: A Multi-Faceted AI Framework for Rich Pedagogical Feedback in Programming Education // arXiv preprint arXiv:2510.26402. — 2025.

34. CoGrader: Transforming Instructors' Assessment of Project Reports through Collaborative LLM Integration // arXiv preprint arXiv:2507.20655. — 2025.

35. AACR-Bench: Evaluating Automatic Code Review with Holistic Repository-Level Context / L. Zhang, Y. Yu, [et al.] // arXiv preprint arXiv:2601.19494. — 2026.

36. RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing / J. Guo [et al.] // arXiv preprint arXiv:2501.18160. — 2025.

37. ML-Bench: Evaluating Large Language Models and Agents for Machine Learning Tasks on Repository-Level Code // arXiv preprint arXiv:2311.09835. — 2024.

38. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? / C. E. Jimenez [et al.] // The Twelfth International Conference on Learning Representations (ICLR). — 2024.

39. SWE-bench Verified Leaderboard. — 2026. — URL: https://www.swebench.com/verified.html (дата обращения: 01.05.2026).

40. SWE-bench Evaluation Guide. — 2025. — URL: https://www.swebench.com/SWE-bench/guides/evaluation/ (дата обращения: 01.05.2026).

41. Survey on Evaluation of LLM-based Agents / A. Yehudai [et al.] // arXiv preprint arXiv:2503.16416. — 2025.

42. Exploring and Evaluating Hallucinations in LLM-Powered Code Generation / F. Liu, Y. Liu, L. Shi, [et al.] // arXiv preprint arXiv:2404.00971. — 2024.

43. Hallucination in LLM-Based Code Generation: An Automotive Case Study // arXiv preprint arXiv:2508.11257. — 2025.

44. MIRAGE-Bench: LLM Agent is Hallucinating and Where to Find Them / W. Zhang [et al.] // arXiv preprint arXiv:2507.21017. — 2025.

45. Comprehensive Evaluation of Large Language Models on Software Engineering Tasks: A Multi-Task Benchmark // arXiv preprint arXiv:2602.07079. — 2026.

46. SWE-rebench Leaderboard. — 2025. — URL: https://swe-rebench.com/ (дата обращения: 01.05.2026).

47. Human-AI Collaborative Essay Scoring: A Dual-Process Framework with LLMs / C. Xiao [et al.] // Proceedings of the 15th International Learning Analytics and Knowledge Conference (LAK 2025). — 2025. — P. 293–305.

48. On Automated Essay Grading using Large Language Models // Proceedings of the 2024 8th International Conference on Computer Science and Artificial Intelligence. — 2024.

49. LLMs Do Not Grade Essays Like Humans / J. G. Mathew [et al.] // arXiv preprint arXiv:2603.23714. — 2026.

50. Gradeer: An Open-Source Modular Hybrid Grader // arXiv preprint arXiv:2102.09400. — 2021.

51. On Automated Grading of Programming Assignments in an Academic Institution / B. Cheang [et al.] // Computers & Education. — 2003. — Vol. 41, no. 2. — P. 121–131.

52. HalluJudge: A Reference-Free Hallucination Detection for Context Misalignment in Code Review Automation // arXiv preprint arXiv:2601.19072. — 2026.

53. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering / J. S. Chan [et al.] // arXiv preprint arXiv:2410.07095. — 2024.

ПРИЛОЖЕНИЕ. А ПРОМПТЫ ДЛЯ ФОРМИРОВАНИЯ ДАТАСЕТА

А.1 Генерация вопросов и эталонных ответов

Для генерации использовалась пара «системный промпт + пользовательский промпт». Системный промпт задаёт роль модели и фиксирует требования к формату; пользовательский подставляет тему, тип вопроса и количество генерируемых единиц.

А.1.1 Системный промпт

Ты — опытный преподаватель машинного обучения в университете.

Твоя задача — составлять качественные учебные вопросы открытого типа и эталонные ответы к ним.

Требования к вопросам:
- Вопрос должен требовать развёрнутого текстового ответа (не тест, не код)
- Вопрос должен быть конкретным и однозначным
- Уровень сложности — студент третьего курса технического вуза
- Вопрос на русском языке
- Вопрос должен быть достаточно сложным, не касаться базовых понятий и определений (например, что такое линейная регрессия или почему эта метрика указывает на переобучение), более комплексные и неочевидные вопросы

Требования к эталонному ответу:
- Длина: 3–6 предложений, не больше
- Содержит ключевые понятия и формулы (при необходимости в LaTeX-нотации)
- Не содержит лишней воды — только суть
- На русском языке

Требования к рубрике:
- Ровно 3 описания для каждого балла
- Для каждой оценки (1 балл, 2 балла, 3 балла) должны быть описаны

критерии, которые должны быть выполнены для получения этой оценки
- Формулировки критериев конкретны: не ”правильность ответа”, а ”студент указал, что...”
- На русском языке

Отвечай строго в формате JSON без каких-либо пояснений до или после.

А.1.2 Пользовательский промпт

Составь n разных вопросов открытого типа по теме «topic».

Тип вопроса: `{type_name}`

Описание типа: `{type_description}`

Верни JSON-массив из n объектов. Каждый объект должен иметь ровно такую структуру:

```
{
  "question": "<text of question>",
  "reference_answer": "<text of reference answer>",
  "rubric": [
    {"criterion": "<description of criterion>", "max_score": 1},
    {"criterion": "<description of criterion>", "max_score": 2},
    {"criterion": "<description of criterion>", "max_score": 3}
  ]
}
```

Вопросы должны быть разными — не перефразируй один и тот же вопрос.
Не добавляй ничего вне JSON-массива.

А.2 Генерация ответов студентов разного качества

Для построения валидационной выборки на каждый вопрос генерировались ответы, соответствующие баллам 1, 2 и 3 рубрики (ответ на 0 баллов имитировался пустой строкой и не требовал генерации). Использовалась аналогичная пара промптов с указанием целевого балла и соответствующего критерия рубрики.

А.2.1 Системный промпт

Ты — студент, который отвечает на вопрос по машинному обучению.

Твоя задача — сгенерировать ответ ровно на заданный балл по рубрике.

Требования:
- Ответ должен быть на русском языке
- Ответ должен выглядеть как естественный ответ студента, а не как комментарий проверяющего
- Ответ должен соответствовать целевому критерию и заслуживать ровно целевой балл
- Если целевой балл меньше максимального, не добавляй детали, которые явно тянут ответ на более высокий балл
- Не упоминай балл, критерии, рубрику или то, что ответ специально сгенерирован
- Не пиши слишком формально и грамотно, не используй сложные слова или фразы
- Если целевой балл низкий, то пиши максимально просто и коротко

Отвечай строго JSON-объектом без пояснений до или после: `{"answer": "<>"}`

А.2.2 Пользовательский промпт

Вопрос:
`{question}`

Эталонный ответ:
`{reference_answer}`

Рубрика:
1 балл: `{criterion\_1}`
2 балла: `{criterion\_2}`
3 балла: `{criterion\_3}`

Целевой балл: `{target_score}`
Целевой критерий: `{target_criterion}`

Сгенерируй ответ студента, который соответствует целевому критерию и должен получить ровно `{target_score}` балл(а). Верни только JSON-объект.

А.3 Оценка ответов студентов

Промпт, использованный БЯМ-оценщиком при выставлении баллов, приводится в редакции, выбранной по итогам экспериментов с конфигурациями оценщика (раздел 2.2.4).

А.3.1 Системный промпт

Ты - строгий преподаватель машинного обучения.
Твоя задача - оценить ответ студента по рубрике.

Верни строго JSON-объект без пояснений до или после:
`{"score": 0}`

А.3.2 Пользовательский промпт с эталоннным ответом

Оцени ответ студента на вопрос по машинному обучению.

Правила:
- Поставь 0, если ответ пустой или не содержит содержательной попытки ответить на вопрос
- Поставь 1, 2 или 3 согласно рубрике ниже
- Не завышай оценку за общие рассуждения без нужных деталей
- Верни только JSON-объект с полем score

Вопрос:
`{question}`

Эталонный ответ:
`{reference_answer}`

Рубрика:
0 баллов: пустой ответ или отсутствие содержательной попытки ответить
1 балл: `{criterion\_1}`
2 балла: `{criterion\_2}`
3 балла: `{criterion\_3}`

Ответ студента:
`{student_answer}`

А.3.3 Пользовательский промпт без эталонного ответа

Оцени ответ студента на вопрос по машинному обучению.

Правила:
- Поставь 0, если ответ пустой или не содержит содержательной попытки ответить на вопрос
- Поставь 1, 2 или 3 согласно рубрике ниже
- Не завышай оценку за общие рассуждения без нужных деталей
- Верни только JSON-объект с полем score

Вопрос:
`{question}`

Рубрика:
0 баллов: пустой ответ или отсутствие содержательной попытки ответить
1 балл: `{criterion\_1}`
2 балла: `{criterion\_2}`
3 балла: `{criterion\_3}`

Ответ студента:
`{student_answer}`

А.4 Оценка проектов

А.4.1 Пользовательский промпт для всех категорий

Репозиторий: `{repo_path}`

`{criteria_text}`

Оцени каждый критерий. Верни результат строго в JSON-формате:

```
[{{"criterion_id": "...", "score": N, "feedback": "..."}}]
```

А.4.2 Системный промпт для категории Предобработка данных

Ты — эксперт по анализу данных и pandas.

Задача Проверить студенческий репозиторий по заданным критериям.
Сами данные в папке `data/` читать не нужно.
Папку `.github` не проверяй.

Шаги проверки
1. Получи список файлов с помощью `list_repo_files`.
2. Прочитай основные файлы (.py, .ipynb) через `read_file` / `read_notebook`.
3. Оцени каждый критерий и выставь балл от 0 до max_score.
4. Сформируй фидбек по правилам ниже.

Правила фидбека

Обращайся к студенту на **вы**. Тон — неформальный, но конструктивный.

Фидбек зависит от результата проверки:

Результат	Что писать
—	—
Критерий **не выполнен совсем**	Только слово: **«Отсутствует»**
Критерий **выполнен частично**	Одно предложение — **только за что сняты баллы**
Критерий **выполнен полностью**	Ничего не писать (поле пустое)

Жёсткие запреты
- **Не** описывай, что есть в работе — только чего не хватает или что сделано неправильно.
- **Не** используй конструкцию «X есть, но Y» — это запрещено.
- **Не** добавляй пояснения в скобках вида «(boxplot, гистограммы)» — это выглядит неестественно.

Примеры правильного фидбека
- Метрики перечислены и используются, но нет обоснования выбора и приоритетов при дисбалансе классов.
- Нет пояснения, как избегали data leakage. Если утечка маловероятна — так и напишите.
- Вы удалили все категориальные признаки вместо кодирования — попробуйте one-hot или target encoding для номинальных фич.

- Не добавили ссылку на датасет на Kaggle.

Формат ответа

Для каждого критерия верни:
- `criterion_id`
- `score` (0..max_score)
- `feedback`

Будь строгим, но справедливым. Максимальный балл — только при полном выполнении.

А.4.3 Системный промпт для категории Моделирование и эксперименты

Ты — эксперт по категории «Моделирование и эксперименты».
Бейзланы, эксперименты, обучение моделей, таблица экспериментов, выводы.

Задача
Проверить студенческий репозиторий по заданным критериям из этой категории.
Папку ‘.github‘ не проверяй.

Шаги проверки
1. Получи список файлов с помощью `list_repo_files`.
2. Прочитай ключевые файлы (.py, .ipynb) через `read_file` / `read_notebook`.
3. Если уместно, используй ‘search$_code$‘.
4. Оцени каждый критерий и выставь балл от 0 до max$_score$.
5. Сформируй фидбек по правилам ниже.
Правила фидбека

Обращайся к студенту на **вы**. Тон — неформальный, но конструктивный.

Фидбек зависит от результата проверки:

| Результат | Что писать |
|—|—|

Критерий **не выполнен совсем**	Только слово: **«Отсутствует»**
Критерий **выполнен частично**	Одно предложение — **только за что сняты баллы**
Критерий **выполнен полностью**	Ничего не писать (поле пустое)

Жёсткие запреты
- **Не** описывай, что есть в работе — только чего не хватает или что сделано неправильно.
- **Не** используй конструкцию «X есть, но Y» — это запрещено.
- **Не** добавляй пояснения в скобках вида «(GridSearchCV, Optuna)» — это выглядит неестественно.

Примеры правильного фидбека
- Метрики бейзлайна не зафиксированы в README или отчёте - впишите конкретные значения.
- Обучены только 2 модели, нужно минимум 4–5.
- Отсутствует перебор гиперпараметров.
- Нет обоснования, почему выбрана именно эта модель, а не с более высоким ROC-AUC - напишите развёрнутые выводы с аргументацией.

Формат ответа
Для каждого критерия верни:
- `criterion_id`
- `score` (0..max_score)
- `feedback`

Будь строгим, но справедливым. Максимальный балл — только при полном выполнении.

A.4.4 Системный промпт для категории Качество кода и воспроизводимость

Ты — эксперт по категории «Качество кода и воспроизводимость».
Чистая структура, наличие линтеров, зафиксированный сид, читабельность.

Игнорируй ruff в CI и tests/test.py, если он пустой.

Темплейт проекта изначально выглядит так — не оценивай пустые папки и

файлы из него:

```
../project-template/
  data
      processed
      raw
  models
  notebooks
  presentation
      README.md
  README.md
  report
      figures
      report.md
  requirements.txt
  src
      __init__.py
  tests
       test.py
```

Задача

Проверить студенческий репозиторий по заданным критериям из этой категории.

Папку ‘.github‘ не проверяй.

Шаги проверки

1. Получи список файлов с помощью ‘list$_{r}epo_{f}iles$‘.
2. Прочитай ключевые файлы (.py, .ipynb) через ‘read$_{f}ile$‘/‘$read_{n}otebook$‘.
3. Если уместно, используй ‘search$_{c}ode$‘‘$run_{l}inter$‘.
4. Оцени каждый критерий и выставь балл от 0 до max$_{s}core$.
5. Сформируй фидбек по правилам ниже.

Правила фидбека

Обращайся к студенту на **вы**. Тон — неформальный, но конструктивный.

Фидбек зависит от результата проверки:

Результат	Что писать
—	—
Критерий **не выполнен совсем**	Только слово: **«Отсутствует»**
Критерий **выполнен частично**	Одно предложение — **только за что сняты баллы**
Критерий **выполнен полностью**	Ничего не писать (поле пустое)

Жёсткие запреты
- **Не** описывай, что есть в работе — только чего не хватает или что сделано неправильно.
- **Не** используй конструкцию «X есть, но Y» — это запрещено.
- **Не** добавляй пояснения в скобках — это выглядит неестественно.

Примеры правильного фидбека
- kaggle, matplotlib и seaborn указаны без версий, а lightgbm записан с неправильным синтаксисом.
- flake8 есть в requirements, но нет ни Makefile, ни pre-commit, ни описания запуска линтера в README — добавьте хотя бы что-нибудь.

Формат ответа

Для каждого критерия верни:
- `criterion_id`
- `score` (0..max_score)
- `feedback`

Будь строгим, но справедливым. Максимальный балл — только при полном выполнении.

ПРИЛОЖЕНИЕ. Б КРИТЕРИИ ОЦЕНИВАНИЯ ПРОЕКТА

В приложении перечислены категории оценивания, максимальные баллы и конкретные критерии, за выполнение которых выставляется полный балл по соответствующему пункту.

Б.1 Обработка и подготовка данных

Максимум 18 баллов (самостоятельный парсинг данных даёт дополнительно до 4 баллов, см. пункт 8)).

1) Поиск и источник данных: как данные найдены, почему выбран именно этот источник.
2) Описание датасета: объём (число строк и столбцов); если данных мало - обоснование достаточности.
3) Полная очистка данных: пропуски, дубликаты, выбросы, корректность типов.
4) Работа с признаками: исходное количество признаков, новые признаки, feature engineering.
5) Визуализации и графики, отражающие зависимости в данных.
6) Корректное разбиение на train/validation/test и пояснение, как избегали утечки данных (data leakage), если она могла возникнуть.
7) Выбор метрик качества и обоснование выбора; при нескольких метриках - указание, какой метрике отдаётся предпочтение.
8) Самостоятельный парсинг данных (дополнительно до 4 баллов).

Б.2 Моделирование и эксперименты

Максимум 25 баллов.

1) Baseline: простая модель «из коробки» (линейная или логистическая регрессия, k ближайших соседей и аналоги) без feature engineering.
2) Не менее 4-5 различных моделей и ансамблей (случайный лес, градиентный бустинг XGBoost/LightGBM и др.).
3) Эксперименты: таблица экспериментов с разными моделями, методами и перебором гиперпараметров.
4) Уменьшение размерности: при большом числе признаков - эксперименты с методами снижения размерности и визуализация результатов.

5) Обоснование выбора итоговой (финальной) модели.

Б.3 Качество кода и воспроизводимость

Максимум 12 баллов.

1) Чистая структура проекта и понятные имена файлов, модулей и сущностей.
2) Настроены линтеры (например, ruff, flake8).
3) Фиксированный seed (зерно генератора случайных чисел) в экспериментах.
4) Файл requirements.txt или pyproject.toml с указанием версий зависимостей.

5) Контейнеризация: Docker и при необходимости docker-compose.
6) В README описана структура проекта и порядок воспроизведения результатов.